
PEEK-Online: Predictive Queue-Informed KV Cache Management for LLM Serving

Abstract

1 We present PEEK, a lightweight scheduling and eviction framework for online
2 LLM serving. PEEK maintains an incremental *radix tree over the pending queue*,
3 exposing prefix-sharing clusters on existing engine surfaces. A low-overhead
4 *dual-walk* matches the tree against the engine’s prefix cache to yield longest-prefix-
5 match for every waiting request; PEEK then admits cluster pioneers first so siblings
6 inherit the freshly cached prefix, a co-designed eviction hook protects blocks an-
7 cestral to queued demand, and a *multi-lane* stride scheduler bounds starvation.
8 On SGLang and vLLM across five workloads up to $4\times H100$ (DP=2 over TP=2),
9 PEEK delivers up to $3.0\times/2.6\times$ cache hit, $7.9\times/7.1\times$ TTFT, $6.7\times/5.5\times$
10 E2E, and $3.6\times/4.5\times$ throughput gains over each engine’s strongest stock base-
11 line (SGLang/vLLM), while matching baselines within noise on workloads with
12 no exploitable prefix structure. Wins hold as KV-cache pressure and inference
13 parallelism scale. Code will be released.

14 1 Introduction

15 Modern LLM serving engines optimize KV cache reuse: SGLang’s RadixAttention [Zheng et al.,
16 2024a] provides longest-prefix matching over cached blocks; vLLM’s PagedAttention [Kwon et al.,
17 2023] enables block-level sharing via OS-style paging. Yet both stop at the cache—neither reasons
18 about which *waiting* requests share prefixes with each other, leaving the predictive signal in the
19 request queue unexploited (Table 1). SGLang’s prefix-aware policies (**LPM**, **DFS_WEIGHT**) remain
20 bound to current cache state and degrade once the cache is cold; vLLM defaults to FCFS, which under
21 memory pressure devolves into *cache thrashing* as interleaved prefix groups evict one another (§2).
22 In both engines, scheduling and eviction are decoupled, precluding joint optimization—inefficiencies
23 magnified in long-context workloads such as RAG and agentic pipelines.

24 We present **PEEK**, a lightweight scheduling and eviction framework for both online (streaming) and
25 offline (batch) LLM serving; *this paper focuses on online*. Our motivation is a simple observation:
26 **the waiting queue is itself a structured workload whose prefix-sharing relationships can directly**
27 **drive KV-cache reuse**. PEEK retains LPM’s prefix-match objective but feeds it richer signals at
28 lower overhead from an incremental radix tree via three mechanisms (Appendix Table 3):

- 29 1. **See the queue’s structure for cheap (§3.1).** PEEK maintains an incremental radix tree over
30 the pending queue—no per-cycle rebuild, $O(D)$ per insert/remove, $O(1)$ per query on schedul-
31 ing/eviction signals. To compute LPM hits against the engine cache, PEEK’s *dual-walk* co-
32 descends the pending tree against SGLang’s radix cache in one pass, returning all per-request hits
33 in $O(C\times D)$ vs. a naive $O(N\times D)$; for high-sharing workloads $C \ll N$, an N/C amortization¹.
34 On vLLM’s flat block-hash cache, PEEK falls back to per-request probes but still reads cluster
35 signals off the tree—structure stock vLLM cannot see.
- 36 2. **Cluster-aware admit and evict (§3.2).** PEEK’s *Cluster-LPM* (cLPM) extends LPM with pending-
37 tree signals on top of its hit-length objective h . Requests in the same cluster therefore rank close
38 together, so one prefix amortizes across the cluster before unrelated arrivals can evict the prefix—
39 instead of scattering by arrival order and thrashing the cache (e.g., stock LPM on hit-length ties;
40 §2). The eviction hook reads the same tree from the other side: zero-demand blocks evict first;
41 blocks the largest pending clusters depend on are protected last.

¹ N = queue size; C = number of prefix-sharing clusters; D = prompt depth in the tree ($\sim O(\log N)$).

	FCFS (vLLM/SGLang)	LPM (SGLang)	DFS_W (SGLang)	PEEK (Ours)
<i>Scheduling</i>				
Prefix-aware Ordering	X Arrival	Cache tree [‡] Per-request	Cache tree [‡] Batch	Queue trie Global
<i>Caching & Eviction</i>				
Mechanism	APC / RadixCache	RadixCache	RadixCache	Pending trie
Eviction	LRU/LFU	LRU/LFU	LRU/LFU	Queue-aware

Table 1: Scheduling and eviction mechanisms in current serving engines. PEEK is the only engine that jointly optimizes scheduling and eviction. [‡]LPM/DFS_W see only cache state, not queue sharing.

3. **Bound starvation, adaptively (§3.2).** We observed that cluster-first ordering can starve singletons. PEEK adds a fairness lane keyed by arrival, stride-interleaved with cLPM, and a *dynamic-lane* controller widens the fairness share as singletons accumulate or as the oldest singleton’s wait nears the SLO (EMA-smoothed). Thus, cluster locality dominates on prefix-rich queues; fairness ramps up only when the queue tilts singleton-heavy.

We evaluate PEEK on five workloads spanning single-GPU and multi-GPU deployments (up to 4×H100, DP=2 over TP=2) across SGLang and vLLM (Appendix Table 6). Our key findings:

- **Substantial wins where prefix structure exists (§4.1–§4.3).** Across shared-prompt chat (W1), long-prefix RAG (W2), and multi-GPU 70B variants of W1 and W2 (W3; 4×H100, DP=2×TP=2), PEEK lifts cache hit **3.0×/2.6×**, TTFT **7.9×/7.1×**, E2E **6.7×/5.5×**, and throughput **3.6×/4.5×** (SGLang/vLLM); gains scale with KV pressure (2–16×) and parallelism.
- **No regression on low-headroom traffic (§4.4).** On agentic conversation bursts already prefix-coherent by arrival (W4) and singleton-dominated chat (W5), PEEK matches the strongest stock baselines within noise on E2E and throughput ($\pm 2\%$ on W4, $\pm 2\%$ on W5; TTFT envelopes are wider, $\pm 5\text{--}7\%$) and modestly wins where weak overlap exists (W5 cell_C_long: +10 pp cache hit, up to **2.6×** TTFT on vLLM)—never regressing.
- **Cluster-aware admission is the dominant lever; other components refine it (§4.1).** cLPM alone lands within **3 pp** cache hit of the full stack. Group-major (GM) shifts cache hit by **−0.1 to +1.4 pp** (SGLang) / **−3.0 to −0.4 pp** (vLLM) but costs **10–29%** TTFT/E2E from singleton deferral; dynamic-lane (DL) fairness recovers **2.4–15.1%**; queue-aware eviction (PE) further trims TTFT/E2E by **0–3%** on SGLang, flat on vLLM.

PEEK also supports the *offline* (batch) regime, resting on the same observation: the pending queue makes future cache demand *predictive*, and the prefix relationships among waiting requests are the highest-signal structure to schedule and evict against. In contrast, PEEK-offline builds a one-shot prefix trie over the full submitted batch, DFS-traverses it to place sharing requests adjacently from a cold cache, and refines wave-level ordering against a shadow LRU model with the same queue-aware eviction hooks (Appendix A.3.1). PEEK-offline delivers up to **1.6×/1.8×** throughput on single-GPU (Qwen2.5-14B/1×H100) and **2.4×/8.0×** on multi-GPU (Qwen2.5-72B/2×H100) over the strongest baselines (SGLang/vLLM), with near-zero overhead on non-sharing workloads (Appendix A.3.2).

2 Background and Motivation

Prefix caching mechanisms. We focus on SGLang and vLLM, the leading open-source LLM serving engines in production use; a *request* is a single completion call (prefill over the input prompt followed by autoregressive decoding). Both engines use *PagedAttention* to decouple logical token sequences from physical KV cache blocks. On top of this, vLLM employs **APC** (Automatic Prefix Caching), which hashes each block by token content and prefix position via chained hashing to identify reusable blocks. SGLang adopts **RadixCache**, organizing cached blocks in a radix tree for $\mathcal{O}(D)$ per-request LPM. These mechanisms provide the *architectural possibility* for prefix sharing; actual performance depends on the *scheduling* and *eviction* policies discussed below.

Scheduling and eviction policies. vLLM uses **FCFS** (arrival order, $\mathcal{O}(1)$, prefix-agnostic); SGLang offers **LPM** (Longest Prefix Match, $\mathcal{O}(N \cdot D)$ per round) and **DFS_WEIGHT**, which groups requests by cache-tree position. All three bind to the *current* cache state and ignore inter-queue sharing: FCFS+APC barely helps under cache pressure (§4.1); LPM (greedy per-round) and DFS_WEIGHT (cache-tree-position grouping) both decide from current cache state and miss waiting-queue sharing.

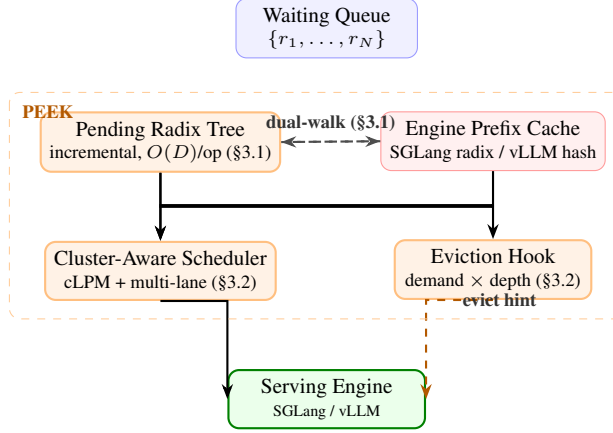


Figure 1: PEEK architecture.

On the eviction side, both engines default to **LRU**, designed for OS page caches with unknown future access—but the LLM waiting queue encodes upcoming work, and LRU may evict prefixes that pending requests need next. SGLang’s **LFU** alternative is similarly queue-oblivious. Under heavy contention, all of these struggle (§4.1).

Scheduling-induced thrashing. Stock LPM ranks by hit length h and breaks ties by arrival—reactive, blind to future queue demand. Consider 8 cold chat requests interleaved across three tenants ($A_1, B_1, C_1, A_2, B_2, C_2, A_3, B_3$; cluster sizes 3/3/2; each tenant’s $\sim 2K$ -token prompt uncached) on a KV cache that holds only one prompt at a time. All tied at $h=0$, LPM admits in arrival order: prefill A , evict for B , evict for C , refill A —**0% hit**. PEEK reorders the queue by cluster via the pending tree to $A_1, A_2, A_3, B_1, B_2, B_3, C_1, C_2$: each prompt prefills once and is reused—**62.5% hit**. At 100+ tenants and $2\text{--}16\times$ KV pressure, the gap widens: cache-state-bound schedulers cannot anticipate which prefix the queue will demand next (§4.1).

The gaps. (i) *Visibility*: existing schedulers ignore inter-queue sharing. (ii) *Coupling*: scheduling and eviction are decoupled. (iii) *Co-design*: scheduling and eviction share the same queue-demand signal but no engine exploits it jointly. PEEK closes all three: an incremental radix tree over the waiting queue exposes prefix-sharing clusters the engines cannot see, driving cluster-aware admission and a co-designed eviction hook that protects blocks queued requests will reuse (§3).

3 PEEK: Predictive Queue-Informed KV Cache Management

PEEK rests on a simple observation: **the waiting queue is itself a structured workload**. To utilize its signals, PEEK introduces an incremental radix tree over the pending queue (§3.1) supporting three mechanisms (Figure 1): (1) a *dual-walk* (§3.1) yielding all-LPM via $O(C\times D)$ tree co-descent on SGLang’s radix cache and per-request hash probes on vLLM’s hash cache; (2) *cluster-aware admission* (§3.2) admits the pioneer first so siblings reuse the cached prefix, paired with an *eviction hook* that frees only blocks that no pending cluster needs; (3) a *multi-lane stride scheduler* (§3.2) interleaving cache-locality with fairness.

PEEK pairs a Rust core (~ 1.6 kLOC; pending tree, dual-walk, cluster/score lookups; PyO3) with ~ 800 LOC Python shims per engine that monkey-patch SGLang’s RadixCache/Scheduler and vLLM v1’s Scheduler/BlockPool; cLPM sort runs in Python over Rust-computed signals. Environment flags toggle PEEK per-binary, no upstream fork (Appendix A.1.2).

3.1 Pending Radix Tree and Dual-Walk for LPM

The pending radix tree is a compressed trie over the waiting prompts. Each leaf carries the request IDs (*terminators*) whose prompt ends there; each internal node holds a `pending_count`—the number of waiting prompts passing through. Edges store multi-token runs via radix compression, so size scales with distinct prefix branches, not raw token count. PEEK maintains the tree incrementally: `insert` on arrival, `remove` on admission/completion, both $O(D)$ along the prompt’s path (typically $O(\log N)$). The tree is always current at scheduling time—no per-cycle rebuild. Two requests share

Algorithm 1 Dual-Walk on SGLang: per-cycle LPM via tree-vs-tree co-descent

Require: Pending tree T ; SGLang radix cache $\mathcal{C}$

Ensure: LPM hit length $H[r]$ for every waiting request r

```
1:  $H \leftarrow \{\}$ ; VISIT( $T.\text{root}$ , 0); return  $H$ 
2: procedure VISIT(pending node  $u$ , matched depth  $d$ ):
3:   for each child edge  $e = (u \rightarrow u')$  of  $u$  do
4:      $\delta \leftarrow \text{MATCHINRADIX}(e.\text{tokens}, \mathcal{C}, d)$  {co-traverse  $\mathcal{C}$  from depth  $d$ }
5:      $h \leftarrow d + \delta$ 
6:     for each terminator  $r$  in  $\text{SUBTREE}(u')$  do
7:        $H[r] \leftarrow \max(H[r], h)$  {LPM along this branch}
8:     end for
9:     if  $\delta = |e.\text{tokens}|$  then
10:      VISIT( $u'$ ,  $h$ ) {full edge matched: descend}
11:    end if
12:  end for
13: vLLM variant. The flat block-hash cache permits no co-descent. Instead, PEEK calls the engine's per-request find_longest_cache_hit for each  $r$  to obtain  $H[r]$  in  $O(N \times b)$  ( $b$ =block-hash chain length), then walks the pending tree separately ( $O(N)$ ) to read cluster signals.
```

121 a queued prefix iff their prompts pass through a common internal node with `pending_count` ≥ 2 ;
122 the deepest such node is the request's *cluster*, and its `pending_count` the cluster size.

123 Each cycle, PEEK obtains the LPM hit $h(r)$ for every waiting request and separately reads its cluster
124 signals (size, ancestor score) from the pending tree at $O(1)$ per request. On SGLang's radix cache,
125 PEEK *dual-walks*—tree-vs-tree co-descent of the pending tree against the radix cache (Algorithm 1;
126 Figure 6 in Appendix A.1.4)—collapsing N per-request lookups into C per-cluster lookups, $O(C \times D)$
127 vs. $O(N \times D)$, an amortization of N/C that is large for high-sharing workloads where $C \ll N$.
128 On vLLM's flat block-hash cache, no structural co-descent is possible: PEEK calls the engine's
129 per-request `find_longest_cache_hit` once per request ($O(N \times b)$, b = block-hash chain length),
130 then reads cluster signals separately from the pending tree—an $O(N)$ traversal exposing the cluster
131 structure stock vLLM cannot see.

132 3.2 Pending-Tree-Driven Scheduling and Eviction

133 PEEK's scheduler (cLPM) and eviction hook (PE) both read the same pending tree, so admission
134 and protection are consistent without explicit coordination. The full per-cycle procedure is given in
135 Algorithm 2 (Appendix A.1.5); we walk through its components below.

136 For every waiting request r , cLPM produces three signals: (1) the *LPM hit* $h(r)$ against the engine
137 cache; (2) the *section* r belongs to—*warm* if $h(r) > \tau$, otherwise *pioneer* if r 's first $\theta=32$ tokens have
138 not yet been claimed this cycle, else *sibling*; (3) two pending-tree statistics, $|\text{cluster}(r)|$ (pending
139 count at r 's deepest ≥ 2 ancestor) and the *ancestor score* $\text{score}(r) = \sum_{v \in \pi(r)} \text{pending_count}(v) \cdot$
140 $|\text{edge}(v)|$ (depth-weighted demand along r 's path). cLPM then sorts by

$$\text{key}(r) = (\text{section}(r), -h(r), -\text{score}(r), -|\text{cluster}(r)|, \text{arrival}(r)). \quad (1)$$

141 By comparison, stock LPM is a special case: it sorts by $(-h(r), \text{is_depr}(r), \text{arrival}(r))$, where
142 `is_depr` is the same θ -prefix pioneer/sibling flag. cLPM differs in two ways: it *promotes* the section
143 distinction from a binary tiebreak to the primary key (so any pioneer admits before any sibling,
144 regardless of h), and it *adds* the cluster-aware tiebreaks `-score` and `-|cluster|` (which LPM
145 cannot compute—it has no pending tree).

146 Against the thrashing failure mode (§2), cLPM's score ordering keeps a larger cluster's members
147 together *within* their section, amortizing each prefill across multiple hits—making cLPM the dominant
148 cache-hit lever (within 3 pp of the full PEEK stack, §4.1). PEEK additionally supports an optional
149 strengthening, *group-major* (GM), that emits all members of one cluster contiguously (1 prefill +
150 `size-1` hits per cluster); cLPM's score ordering already approximates GM grouping in typical
151 queues, so GM adds only marginal cache hit.

152 We observed that GM helps cache hit but aggressively defers singletons. To rescue them, PEEK
153 introduces a *fairness lane* and a *dynamic-lane* (DL) controller: cLPM stride-interleaves Lane A (Eq. 1)

Policy labels		
Label	Components (scheduler + eviction)	Role
FCFS+LRU	FCFS + LRU	Naïve baseline (no prefix caching)
LPM+LRU	SGLang LPM + LRU	SGLang baseline
FCFS(APC)+LRU	vLLM FCFS + APC + LRU	vLLM baseline
LPM+PE / FCFS(APC)+PE	stock scheduler + PEEK PE	Eviction-only ablation
cLPM	cLPM + LRU	Sort-key-only ablation
cLPM+GM	cLPM + group-major + LRU	Scheduling (sort + admission)
cLPM+GM+PE	cLPM + group-major + PEEK PE	Primary co-design
cLPM+GM+DL+PE	cLPM + group-major + dynamic-lane + PE	+ starvation guard

Table 2: Policy labels used in Section 4.

with the fairness lane keyed by (section, arrival, $-h$): a fraction α of admissions come from Lane A , $1-\alpha$ from Lane B . The oldest waiting request advances at least every $\lceil 1/(1-\alpha) \rceil$ cycles. Default $\alpha=0.7$. DL adjusts α per cycle from queue composition—`singleton_frac` (fraction in no cluster) and `age_pressure`— $\min(1, \text{oldest_singleton_wait}/2s)$ —ramping the fairness share $1-\alpha$ from 0.15 (cluster-dominated) toward 0.60 (singleton-heavy or near-SLO), EMA-smoothed:

$$1-\alpha \leftarrow \text{EMA}_{\beta=0.3}(\text{clamp}(0.15 + 0.5 \cdot \text{singleton_frac} + 0.3 \cdot \text{age_pressure}, 0.10, 0.60)).$$

For eviction, PEEK’s hook (PE) overrides LRU/LFU when the engine must free memory, using the same pending tree: candidate block b with tokens t_b scores

$$\text{evict_score}(b) = \max_{v \in \text{ancestors}(t_b)} \text{pending_count}(v) \cdot \text{depth}(v), \quad (2)$$

depth-weighted demand along b ’s ancestor chain (max captures deep lightly-shared prefixes sitting above heavily-shared subtrees); lowest score evicts first. Of four scoring modes (*plain*, *cluster*, *recency*, *decay*), experiments use *cluster*. SGLang installs a custom `EvictionStrategy`; vLLM patches `BlockPool.get_new_blocks` to prefer zero-demand victims (capped at $4N$).

In summary, DL’s clamp $[0.10, 0.60]$ keeps cache locality dominant (Lane A retains at least 40%) while EMA damps oscillation, rescuing singletons GM defers; PE protects what waiting clusters need by evicting zero-demand blocks first and keeping high-demand blocks last. Together, scheduling and eviction stay aligned through the same pending tree.

3.3 Guards

PEEK’s per-cycle cost (dual-walk + sort) would otherwise apply even when the queue has no sharing. The pending tree exposes a `has_sharing` primitive—an $O(\# \text{root-children})$ scan returning true iff some root-child has `pending_count` ≥ 2 —available for callers to short-circuit cLPM. In addition, on SGLang the stock scheduler’s queue-depth fallback (LPM degenerates to FCFS once the waiting queue exceeds 128 requests) provides a second cheap exit. Empirically these two paths together leave PEEK matching FCFS within $\pm 1.5\%$ on workloads with no exploitable structure (W4, W5; §4.4); the dual-walk + sort cost is amortized to the noise floor in those regimes.

4 Experiments

We evaluate PEEK on a five-workload suite (W1–W5) covering typical production LLM serving scenarios across diverse models, datasets, and single- and multi-GPU hardware (Appendix Tables 4–6, 9). Each workload comprises one or more *cells*—parameter configurations along its primary axis—each run at *moderate* and *heavy* load.² We report means across 3 seeds after ~ 10 – 20% warmup; Table 2 lists policy labels. W1 (§4.1) sweeps the full ablation lattice and identifies cLPM+GM+DL+PE as PEEK’s most performant policy; W2 (§4.2), W3 (§4.3), and W4–W5 (§4.4) probe orthogonal axes using cLPM+GM+DL+PE alone, conserving GPU resources.

²Moderate targets sustained queue length 60–127; heavy targets ≥ 128 , the threshold at which SGLang’s LPM falls back to FCFS. The same targets are used on vLLM for cross-engine comparability.

4.1 W1: Shared-Prompt Chat Across Production KV-Pressure Tiers

W1 tests whether PEEK’s co-design holds up as KV-cache pressure rises. Cells A/B/C/D span $2\times$ to $16\times$ KV oversubscription, each mapped to a production class (Appendix Tables 7–8). Figures 2–3 plot cache hit and TTFT; throughput and E2E are in Appendix Figures 8, 9 and full numerical results in Appendix Tables 10–12. We first use cLPM+GM+DL+PE to compare against the stock baselines.

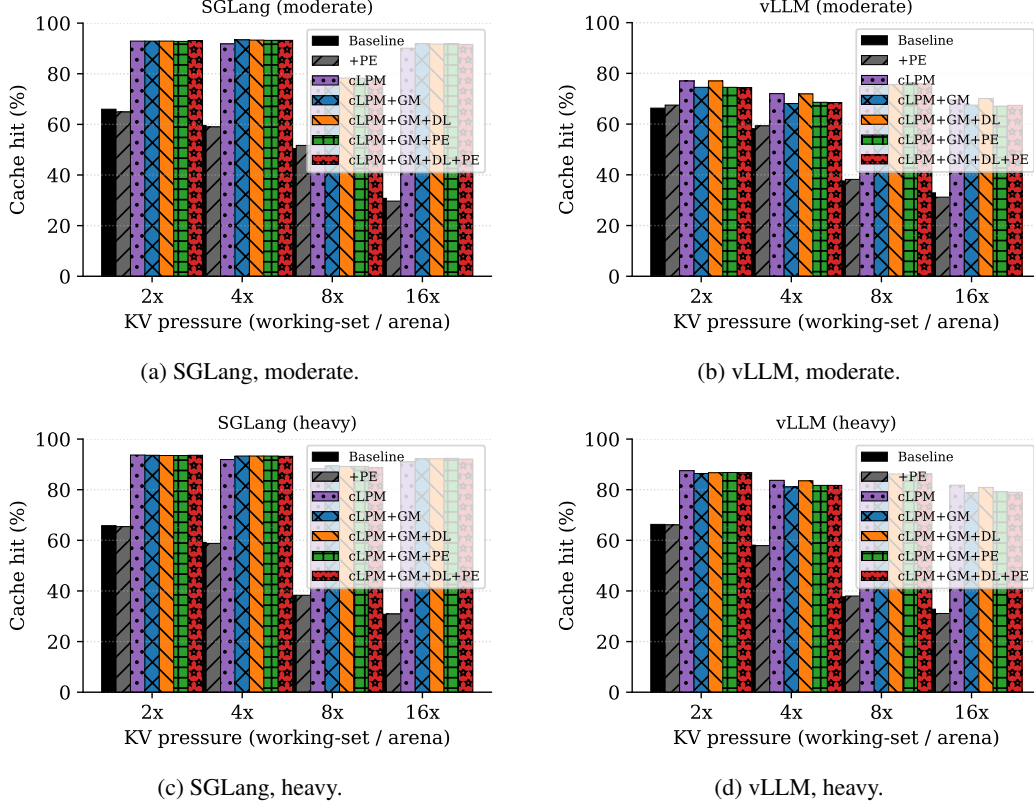


Figure 2: W1 cache hit rate vs. KV pressure.

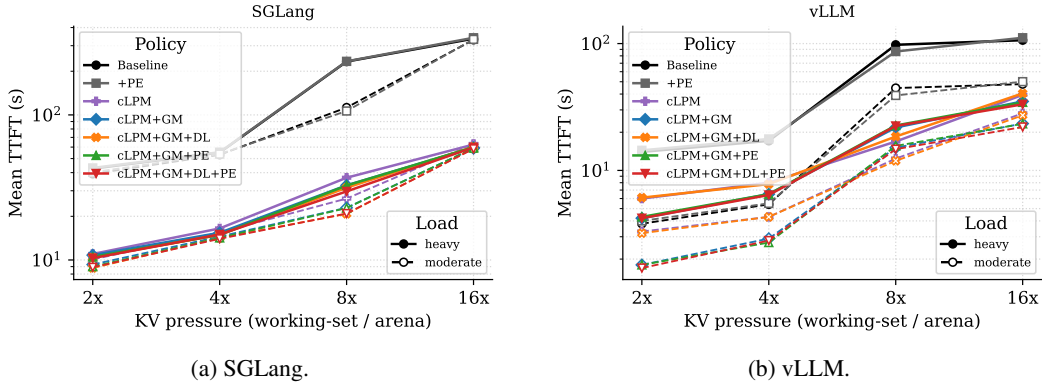


Figure 3: W1 mean TTFT vs. KV pressure (log y-axis). Each panel overlays both load levels: solid lines with filled markers = heavy, dashed lines with hollow markers = moderate.

cLPM+GM+DL+PE lifts cache hit by 28–61 pp (SGLang, heavy) / 20–49 pp (vLLM, heavy; moderate cells start as low as 8 pp)— $1.4\text{--}3.0\times/1.3\text{--}2.4\times$ —and these gains transfer to TTFT $3.7\text{--}7.9\times/1.9\text{--}4.4\times$, E2E $2.9\text{--}6.7\times/1.2\text{--}3.6\times$, and throughput $1.5\text{--}3.6\times/1.2\text{--}2.5\times$ (SGLang/vLLM). Gains scale with KV pressure as baselines thrash (§2) while cLPM+GM+DL+PE stays flat, except at $16\times$ heavy load where over-saturation narrows the ratio. Decode-side (Appendix Table 12): TPOT runs **8–26%** (4–12 ms/token) above baseline; ITL tracks TPOT closely (drift +3.6 to +11.5 ms across cells) and moves with throughput rather than per-token cost—both costs are dwarfed by the E2E gains.

cLPM alone contributes most of PEEK’s gain—within **3 pp** of the full PEEK stack on cache hit on both engines—confirming that cluster-aware admission is the dominant lever among PEEK’s mechanisms. The mechanism behind the gain: by replacing stock LPM’s FCFS-on-ties with pending-tree-aware tiebreaking, cLPM admits cluster siblings adjacent to their pioneer, so each shared-prefix prefill is amortized across many cache hits before any unrelated arrival can evict the prefix.

cLPM+GM lifts SGLang cache hit **−0.1 to +1.4 pp** over cLPM but trails the best on TTFT/E2E by up to **10%/8%**; on vLLM it *reduces* cache hit by **0.4–3.0 pp** and trails on TTFT/E2E by up to **29%/22%** at cell C. **GM, aggressive clustering, buys little cache hit but inflates TTFT/E2E**: cLPM already heads the queue with cluster pioneers, and GM locks in the entire cluster as a contiguous batch—amortizing the shared prefill across siblings while deferring singletons.

DL shortens TTFT/E2E by **2.4–15.1%** on SGLang A/C and vLLM C, **recovering the singleton-starvation cost that GM imposes**; elsewhere it trades a small TTFT regression for queue-aging fairness. PE adds **0–3%** on SGLang and is flat on vLLM ($\pm 5\%$); on a stock scheduler PE is flat (± 1 pp, $\pm 2\%$)—**eviction cannot create temporal locality the scheduler did not first establish**.

Takeaway 1. For high-sharing workloads, all cLPM-based PEEK policies substantially outperform stock baselines on both engines on cache hit, TTFT, E2E, and throughput, with the gain scaling as KV pressure rises. The biggest gain comes from cLPM’s cluster-aware scoring atop stock LPM. GM further pushes cache hit but at the cost of delaying singletons, causing a noticeable loss on TTFT and E2E. DL and PE recover those two ends, ultimately establishing cLPM+GM+DL+PE as the most performant policy on both engines.

4.2 W2: Long-Document RAG under Decode-Dominant KV Pressure

W2 tests PEEK on long-prefix RAG, where documents overflow the KV arena (Appendix Table 6). Figure 4 plots cache hit and TTFT; throughput and E2E are in Appendix Figures 10, 11 and full numerical results in Appendix Tables 13–14.

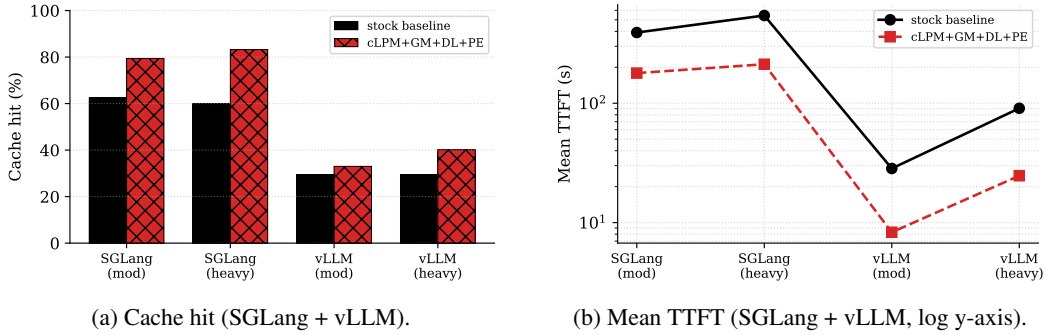


Figure 4: W2 cache hit and mean TTFT, both engines and load levels.

cLPM+GM+DL+PE substantially outperforms the per-engine baseline on long-prefix RAG. Cache hit lifts **+17/+23 pp** (SGLang, mod/heavy) and **+3.6/+10.6 pp** (vLLM)—equivalently **1.27×/1.39×** and **1.12×/1.36×**. These translate into TTFT reductions of **2.2×/2.6×** on SGLang and **3.4×/3.7×** on vLLM, mean E2E reductions of **2.0×/2.4×** and **1.6×/2.3×**, and throughput rising **1.14×/1.26×** on SGLang and **1.00×/1.06×** on vLLM. The large TTFT and E2E reduction (**2×–3.7×**) on both engines reflects the same pattern as W1: under thrashing, low cache hit both inflates per-request prefill and blocks the queue, so cLPM+GM+DL+PE accelerates admitted requests *and* clears the queue faster (§2). Decode-side, mean TPOT moves within $\pm 5\%$ on SGLang and $\pm 9\%$ on vLLM; mean ITL agrees with TPOT to within **1 ms** in every cell—the gain comes from prefill recovery rather than decode-side optimization.

Takeaway 2. As in W1, cLPM+GM+DL+PE wins large on both engines, confirming **PEEK’s advantage transfers to long-decode RAG scenarios**.

4.3 W3: Multi-Cell Shared-Prompt Workloads (Multi-GPU, DP=1 and DP=2)

W3 scales W1 and W2 to Llama-3.1-70B-Instruct (TP=2) at two data-parallel scales—DP=1 (single replica, 2×H100) and DP=2 (two replicas, 4×H100)—across two regimes: *cell B* mirrors W2 (RAG-

like, decode-bound) and *cell C* mirrors W1 (chat-like, admission-bound); see Figure 5, Appendix Figures 12–13 (§A.2.8), Appendix Tables 15–16 (§A.2.7), and Appendix Table 6.

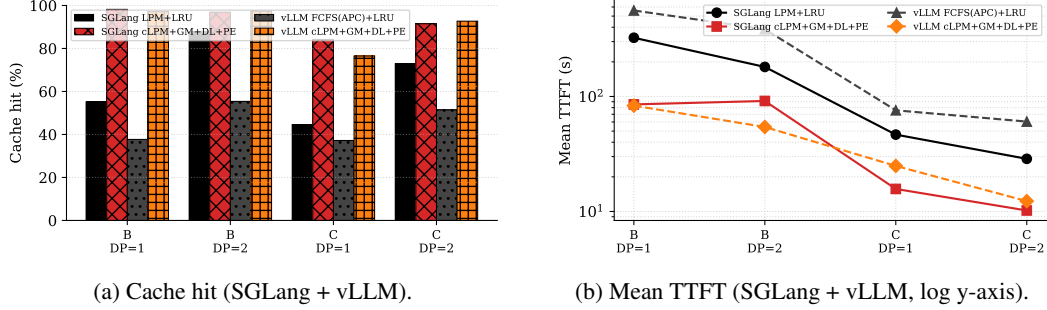


Figure 5: W3 cache hit and mean TTFT across two cells (B, C) and two topologies (DP=1 / 2×H100, DP=2 / 4×H100), heavy load.

cLPM+GM+DL+PE wins large at *both* topologies. At DP=1, SGLang cache hit lifts +43.1/+39.6 pp on cell B/C, TTFT 3.80×/2.97×, E2E 3.62×/2.65×, throughput 2.66×/2.08×; vLLM cache hit lifts +59.4/+39.3 pp, TTFT 6.73×/3.04×, E2E 5.54×/2.58×, throughput 4.45×/2.07×. Doubling to DP=2 mostly helps the stock baseline: SGLang baseline cache hit jumps +32.7/+28.3 pp from DP=1 to DP=2 (vLLM +17.7/+14.2 pp), while PEEK is already at ceiling on cell B (96.9–98.2% SGLang, 97.1–97.3% vLLM) and reaches 84–92% on cell C. The relative gap thus narrows on SGLang to +9.1/+18.6 pp (still up to 2.81× TTFT, 1.62–1.76× throughput) but stays nearly flat on vLLM (+41.9/+41.3 pp; up to 7.08× TTFT, 4.50×/2.83× throughput)—APC’s block-hash misalignment doesn’t shrink with KV space. Notably, PEEK at DP=1 matches or beats the stock baseline at DP=2 on cell B for both engines (SGLang: 98.2 vs 87.8% cache, 85 vs 181 s TTFT, 1.24 vs 1.04 req/s; vLLM: 97.1 vs 55.4%, 83 vs 384 s, 1.07 vs 0.48 req/s)—PEEK delivers more on half the GPUs than the per-engine baseline does on four. Decode-side, TPOT moves within ±7% on SGLang (cell C/DP=1 the worst at −6.9%); on vLLM TPOT rises with throughput on PEEK runs (larger decode batch inflates per-token time in absolute terms, but prefill dominates and E2E still drops sharply).

Takeaway 3. cLPM+GM+DL+PE wins large at both DP=1 and DP=2 on Llama-3.1-70B. PEEK’s gain scales to larger models and multi-GPU topologies.

4.4 No-Regress on Prefix-Coherent and Singleton-Dominated Traffic (W4, W5)

W4 and W5 establish PEEK’s no-regress claim from two angles. W4 runs an already-prefix-coherent scenario (*agentic_only*: multi-turn bursts; *agentic_shared*: 1402-token shared system prompt), where session-burst adjacency leaves little room to improve. W5 runs heterogeneous singleton chat in *cell_C_long* (512–4096 tokens, weak residual overlap) and *cell_C_short* (32–1024 tokens, no structural sharing). Full results in Appendix §A.2.9–A.2.10 (W4) and §A.2.12–A.2.11 (W5).

W4 (already prefix-coherent). cLPM+GM+DL+PE matches or marginally beats the strongest stock baseline (LPM+LRU on SGLang, FCFS(APC)+LRU on vLLM) on every metric in every cell: SGLang cache hit +0.3–1.0 pp, TTFT within ±5%, E2E −0.8–2.5%, throughput +1.1%; vLLM cache hit +0.0–1.7 pp, TTFT within ±7%, E2E and throughput within ±2%. TPOT moves −0.3–3.5% (SGLang) / ±2% (vLLM). Marginal wins land where arrival order mismatches prefix order: SGLang on *agentic_shared*, vLLM on *agentic_only* (APC hash collisions).

W5 (singleton-dominated). On *cell_C_long*, cluster-aware sort detects weak overlap LPM’s exact match misses: SGLang cache hit +9.5 pp (0.7%→10.2%), TTFT −11%, E2E −8%, throughput +5%; vLLM TTFT 2.6× reduction, E2E −20%, throughput +21% (APC reuse 97.5%→98.8%). On *cell_C_short*, the *has_sharing* guard (§3.3) short-circuits PEEK to FCFS: every metric within ±2% except SGLang TTFT +1.7% and vLLM TTFT −10% relative (0.04 s absolute).

Takeaway 4. PEEK is no-regress on W4 and W5: the *has_sharing* guard short-circuits to FCFS when no structure exists, while cluster-aware admission captures residual overlap when it does. Net: within ±2% (W4) and ±1% (W5) of the strongest baseline, with modest wins on weak structure—complementing W1–W3, where PEEK *recovers* locality the baseline destroyed.

5 Related Work

Prefix-aware request scheduling. Online: Preble [Srivatsa et al., 2025] co-optimizes KV reuse with distributed load balancing across replicas. Fu et al. [2024] reorder by predicted output length to reduce head-of-line blocking. DLPM [Cao et al., 2025] adds a delay-bound fairness contract on top of LPM. Dexter et al. [2025] prove the prefix-aware scheduling problem NP-hard and propose a k -LPM approximation. HotPrefix [Li et al., 2025] drives admission and CPU/GPU promotion from a moving average of past hits (*cache hotness*). SOLA [Hong et al., 2025] arbitrates prefix grouping against per-request SLOs. Autellix [Luo et al., 2025] reorders at the program level for agent workloads, capturing inter-call dependencies. Offline / batch: BatchLLM [Zheng et al., 2024b] groups requests by global prefix sharing for high-throughput batched inference. BlendServe [Zhao et al., 2026] reorders an offline workload to jointly maximize prefix/KV reuse and balance compute- vs. memory-bound phases, exploiting full visibility into the request set before submission. PEEK differs by forming groups from the *pending queue itself* via an incremental prefix trie—robust to cold, stale, or contended cache—and uses the same trie for eviction so the two co-optimize one signal; it is intra-replica and composes beneath cross-replica routers, fairness contracts, or SLO arbiters. PEEK-offline (Appendix A.3) sits in the BatchLLM/BlendServe regime; this paper targets the online case with no full-workload visibility.

KV cache reuse across prompts. A separate axis supplies the *capability* to share KV across prompts: PromptCache [Gim et al., 2024] (module-level KV via positional re-encoding), RAGCache [Jin et al., 2024] (hierarchical RAG cache), ChunkAttention [Ye et al., 2024] (attention over a prefix tree), Hydragen [Juravsky et al., 2024] (fused shared-prefix kernel), CachedAttention [Gao et al., 2024] (session-scoped state), CacheBlend [Yao et al., 2024] (block reuse across retrieved chunks), CacheGen [Liu et al., 2024] (compressed KV streaming), LMCache [LMCache contributors, 2025] (distributed shared KV). These are mechanisms for *how* KV is stored; PEEK is the policy for *which* blocks are cached and in what order requests run, composing atop any of them.

Within-sequence KV compression and eviction. H2O [Zhang et al., 2023] keeps heavy-hitter tokens by attention history; SnapKV [Li et al., 2024] clusters and prunes tokens at prefill; Scissorhands [Liu et al., 2023] drops decayed-attention tokens; InfiniGen [Lee et al., 2024] offloads cold KV with prefetch; KIVI [Yuan et al., 2024] quantizes KV to 2-bit. These act per-request along the sequence dimension; PEEK acts across requests along the block-allocation dimension and is fully orthogonal.

Disaggregated and distributed serving. DistServe [Zhong et al., 2024] and Splitwise [Patel et al., 2024] disaggregate prefill from decode; Mooncake [Qin et al., 2025] extends this to a distributed KV pool; Llumnix [Sun et al., 2024] migrates running requests across instances. These reshape *where* requests run; PEEK reshapes *when* they run within a replica, and the two axes compose.

Cross-replica prefix-affinity routing. Gateways such as llm-d [Red Hat and IBM and Google contributors, 2025] score backends by prefix-affinity, queue depth, and KV utilization to route requests to the replica most likely to hit warm cache. This is cross-replica; PEEK improves intra-replica scheduling and sits beneath such a router.

Future-aware cache management. KVFlow [Pang et al., 2025] predicts future cache needs from workflow-graph structure; Marconi [Pan et al., 2025] forecasts reuse from prior traffic. PEEK shares the goal but derives demand from the live pending queue via direct reference counting—no execution graph, no learned forecast—and is complementary across within-tick (PEEK) and across-tick (KVFlow/Marconi) horizons.

6 Conclusion

PEEK-Online treats the waiting queue as a first-class signal for KV cache management via an incremental radix tree, a dual-walk yielding all-LPM in $O(C \times D)$, cluster-aware admission/eviction, and an adaptive fairness lane. Across five workloads on SGLang/vLLM up to $4 \times H100$, PEEK lifts cache hit $3.0 \times / 2.6 \times$, TTFT $7.9 \times / 7.1 \times$, E2E $6.7 \times / 5.5 \times$, throughput $3.6 \times / 4.5 \times$, matches baselines on no-sharing traffic, and composes beneath cross-replica routers and SLO arbiters atop any prefix-reuse mechanism. *Limitations:* the N/C amortization shrinks toward 1 in singleton-dominated traffic (the `has_sharing` guard handles this regime); the cLPM sort still runs in Python; evaluation is on H100 only, leaving smaller-memory accelerators and multi-modal prefixes for future work. Code will be released.

References

- Shiyi Cao, Yichuan Jiang, Zhiqiang Xie, and Lianmin Zheng. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.
- Gregory Dexter, Sen Ye, Aditya Shanbhag, Ion Stoica, and Hao Zhang. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Yichao Fu, Siqi Xue, Yongji Dai, Silei Qiu, Xinyi Sui, Ion Stoica, and Hao Zhang. Efficient LLM scheduling by learning to rank. *Advances in Neural Information Processing Systems*, 37, 2024.
- Bin Gao, Zhuomin Yin, Yifeng Deng, Jianhui He, Zhengda Liu, Zhenzhong Sun, Peng Li, and Yu Cheng. Cost-efficient large language model serving for multi-turn conversations with Cache-dAttention. 2024.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 2024.
- Yuxin Hong et al. SOLA: Optimizing SLO attainment with state-aware scheduling for LLM serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Lam, Huanchen Mao, and Meng Li. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.
- Jordan Juravsky, James Bradbury, Jascha Sohl-Dickstein, and Behnam Neyshabur. Hydragen: High-throughput LLM inference with shared prefixes. *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- Wonbeom Lee, Jungi Lee, Junghwan Seo, and Hwangyu Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. 2024.
- Hao Li, Yi Gu, Jun Huan, et al. HotPrefix: Hotness-aware KV cache scheduling for LLM serving. *Proceedings of the ACM on Management of Data (PACMOD)*, 2025.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37, 2024.
- Yuhan Liu, Hanchen Li, Kuntai Du, Jiayi Yao, Yihua Cheng, Yuyang Huang, Shan Lu, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2023.
- LMCache contributors. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.
- Michael Luo et al. Autellix: An efficient serving engine for LLM agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- Rui Pan et al. Marconi: Prefix caching for the era of hybrid LLMs. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- Shizhe Pang, Yingchao Zhu, Mengwei Huang, Yubin Zeng, Jiaqi Xu, Siyuan Wen, Dongming Zhang, Zhenghua Cai, and Quan Chen. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *Advances in Neural Information Processing Systems*, 38, 2025.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *51st International Symposium on Computer Architecture (ISCA)*, 2024.
- Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Yang, Weimin Zheng, and Zhengyu Zha. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *USENIX Conference on File and Storage Technologies (FAST)*, 2025.

Red Hat and IBM and Google contributors. llm-d: Kubernetes-native distributed inference with KV-cache-aware routing, 2025. Inference Gateway / Endpoint Picker; <https://llm-d.ai>.

Vikranth Srivatsa, Zijian Ding, Zhenyu Wang, Yicheng Hu, Cedric Xia He, Yuan Tian, and Minlan Wang. Preble: Efficient distributed prompt scheduling for LLM serving. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

Biao Sun, Ziming Huang, Hanyu Chen, Jiaming Shi, Wenli Chen, Xingjian Liao, Zhenming He, and Yongji Xiao. Llumnix: Dynamic scheduling for large language model serving. 2024.

Jiayi Yao, Hanchen Li, Yuhang Liu, Siddhant Ray, Yihua Cheng, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

Lu Ye, Ze Dai, Hao Cao, and Erwei Chen. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. *arXiv preprint arXiv:2402.15220*, 2024.

Zirui Yuan, Jingbo Shang, Yang Song, Yifei Wu, Peng Qi, and Albert Gu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

Yilong Zhao, Shuo Yang, Kan Zhu, Lianmin Zheng, Baris Kasikci, Yifan Qiao, Yang Zhou, Jiarong Xing, and Ion Stoica. BlendServe: Optimizing offline inference with resource-aware batching. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2026.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Yineng Liang, Siyuan Zhuang, et al. SGLang: Efficient execution of structured language model programs. In *Proceedings of the 2024 Conference on Neural Information Processing Systems (NeurIPS)*, 2024a.

Zhen Zheng, Xin Han, Zhiyu Yang, Shulin Wu, Fan Zhang, Jianbo Zhu, Mofan Yang, Daoguang Zha, Jian Liu, Gaofeng Yang, and Lixin Zhang. BatchLLM: Optimizing large batched LLM inference with global prefix sharing and throughput-oriented token batching. *arXiv preprint arXiv:2412.03594*, 2024b.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

419 A.1 PEEK: Predictive Queue-Informed KV Cache Management Mechanisms

420 A.1.1 PEEK vs. SGLang LPM: Mechanism Comparison

421 PEEK keeps stock LPM’s longest-prefix-match objective and reuses its pioneer/sibling threshold
 422 ($\theta=32$ tokens), but otherwise replaces every other component with a pending-tree-driven counterpart.
 423 Table 3 lays out the differences axis by axis; each row maps to a section of the main text or this
 424 appendix.

425 A.1.2 Implementation Details

426 **Native core.** The pending radix tree, dual-walk, and per-rid cluster/score lookups are imple-
 427 mented in ~ 1.6 kLOC of Rust (`src/tree.rs`, `src/pending.rs`) and exposed to Python via PyO3
 428 (`peek._core`). Nodes live in a contiguous arena indexed by `NodeId`, with `FxHashMap` children keyed
 429 by the first edge token, `FxHashSet` terminators, and per-node `pending_count`; insert/remove
 430 run in $O(D)$ with no allocation in the hot path. The cLPM sort itself runs in Python ($O(N \log N)$
 431 over the Rust-computed signals); pushing the tree, dual-walk, and per-rid score/cluster computations
 432 into Rust eliminates the per-compare Python callback overhead that dominates stock SGLang LPM
 433 at large N .

Axis	SGLang LPM (stock)	PEEK (cLPM + PE + DL)
Prefix signal source	Engine-side radix cache only — <i>reactive</i> : ranks by what is currently cached	Engine cache <i>plus</i> a pending-queue radix tree — <i>predictive</i> : ranks by what waiting requests will share (§3.1)
Scheduler sort key	$(-h(r), \text{is_depr}(r), \text{arrival}(r))$	$(\text{section}(r), -h(r), -\text{score}(r), - \text{cluster}(r) , \text{arrival}(r))$ (§3.2)
Cold-tie behavior ($h=0$)	Falls back to arrival (FCFS) — interleaves clusters, induces thrashing (§2)	Largest-cluster pioneer first; siblings sort directly behind, so one prefill amortizes across the whole cluster
In-batch deprioritization	Auxiliary radix tree rebuilt from scratch each scheduling call (throw-away)	Read off the persistent pending tree at $O(1)$ per request — no rebuild
Per-cycle LPM cost	$O(N \times D)$ per-request cache lookups	$O(C \times D)$ via tree-vs-tree dual-walk on SGLang ($C = \#$ distinct shared prefixes, $C \ll N$ in high-sharing queues); reuses engine probe on vLLM (§3.1)
Cluster signals exposed	None	Cluster identity, cluster size, depth-weighted ancestor demand score
Eviction policy	LRU/LFU on cache state only — decoupled from scheduling	Queue-aware hook scoring blocks by $\max_{v \in \text{ancestors}} \text{pending_count}(v) \cdot \text{depth}(v)$: zero-demand blocks first, blocks the largest pending clusters depend on protected last (§3.2)
Starvation handling	None beyond the arrival tiebreak	Stride-interleaved fairness lane ($\text{section}, \text{arrival}, -h$); a dynamic-lane controller widens the fairness share with singleton fraction and oldest-singleton wait, EMA-smoothed (§3.2)
No-sharing fallback	Always pays the LPM/aux-tree cost	<code>has_sharing</code> guard short-circuits to FCFS at zero overhead when the queue contains no shared prefixes (§3.3)
Scheduling/eviction coupling	Decoupled; eviction cannot anticipate scheduler choices	Both consume one shared signal (the pending tree) — co-designed without explicit coordination

Table 3: PEEK vs. stock SGLang LPM, axis by axis. PEEK preserves LPM’s longest-prefix-match objective and pioneer/sibling threshold but replaces FCFS tiebreaks with cluster-aware ordering, eliminates the per-cycle aux-tree rebuild via the persistent pending tree, drives queue-aware eviction off the same tree, and bounds singleton starvation with an adaptive fairness lane.

434 **SGLang integration (~814 LOC of Python).** `peek.engines.sglang.patch_hook`
435 monkey-patches two surfaces at import time. `RadixCache.__init__` is patched
436 to install `PeekDemandStrategy` (§3.2) in place of the default heap-based eviction.
437 `Scheduler.process_input_requests` is patched to diff-sync the pending tree against
438 `self.waiting_queue` on every scheduling iteration—a mutation-path-agnostic sync that tolerates
439 any downstream queue manipulation (append, pop, list-comprehension reassignment) without further
440 hooks. The same hook overrides `SchedulePolicy.calc_priority` (the entry point that internally
441 calls `_sort_by_longest_prefix` in stock SGLang) to invoke PEEK’s cLPM sort.

442 **vLLM integration (~800 LOC of Python).** `peek.engines.vllm.patch_hook` hooks
443 three v1 scheduler entry points: `Scheduler.schedule` (sync + reorder `self.waiting`),
444 `Scheduler.add_request` (sync-on-arrival fast path), and `Scheduler.finish_requests` (drop
445 completed rids). Because vLLM’s prefix cache is hash-keyed rather than tree-structured, PEEK
446 additionally maintains an inverted index `block_hash → demand` alongside the pending tree, and
447 patches `BlockPool.get_new_blocks` to scan the free-block queue (capped at $4N$ entries) and
448 prefer zero-demand victims over the default LRU pick. Recency/decay eviction modes also patch
449 `BlockPool.touch` and `cache_full_blocks` to fabricate per-block timestamps that vLLM does
450 not track natively. vLLM v1 spawns its `EngineCore` via `multiprocessing.spawn`, so monkey-

451 patches in the parent do not inherit; PEEK ships a `sitecustomize.py` shim that re-imports the hook
 452 in every child interpreter.

453 **Activation surface.** A single environment flag selects scope: `PEEK_SCHEDULER` for cluster-
 454 aware scheduling, `PEEK_EVICTION` for queue-aware eviction, `PEEK_FLPM` for the multi-lane sched-
 455 uler, `PEEK_EVICTION_MODE` $\in \{\text{plain, cluster, recency, decay}\}$ for the eviction variant, and
 456 `PEEK_FLPM_BIGLANE_SHARE` for the cache-locality lane share (default 0.7). With no flags set, both
 457 shims are no-ops and the engine runs unmodified. Because integration is by monkey-patching, no
 458 fork or PR against either upstream engine is required, and PEEK tracks SGLang’s main branch and
 459 vLLM 0.19.1 with byte-identical baselines.

460 A.1.3 Pending Radix Tree: Build and Clustering

461 The pending radix tree (§3.1) is a textbook compressed trie with two augmentations: per-node
 462 `pending_count` and per-leaf terminators. `INSERT` descends from the root following matching
 463 edges, splitting any edge on token divergence (creating a new internal node) and adding a new leaf
 464 for the request, then bumps `pending_count` along the root-to-leaf path— $O(D)$, where D is the
 465 prompt’s tree depth. `REMOVE` reverses: walk to the request’s terminator, remove the rid, decrement
 466 `pending_count` along the path, and prune empty branches—also $O(D)$. `CLUSTER` walks the
 467 request’s prompt down the tree and returns the deepest node v with `pending_count(v)  $\geq 2$` ; cluster
 468 size is `pending_count(v)`. The tree therefore costs $O(N \cdot D)$ per cycle in total, dominated in
 469 practice by the scheduler’s $O(N \log N)$ sort (§3.2).

470 A.1.4 Dual-Walk for Longest Prefix Match

471 Figure 6 contrasts the two engines. On SGLang’s radix cache, PEEK runs the recursive dual-walk:
 472 tree-vs-tree co-descent collapses N per-request lookups into C per-cluster lookups (Algorithm 1). The
 473 recursion descends a pending edge only when the edge matches in full—a partial match terminates
 474 the branch, since deeper nodes cannot extend the cached prefix. On vLLM’s flat block-hash cache, no
 475 structural co-descent is possible: PEEK calls the engine’s per-request `find_longest_cache_hit`
 476 once per waiting request to obtain $h(r)$, then walks the pending tree separately to read cluster size
 477 and ancestor score—the LPM lookup is not amortized, but the cluster signal stock vLLM’s FCFS
 478 lacks is still surfaced.

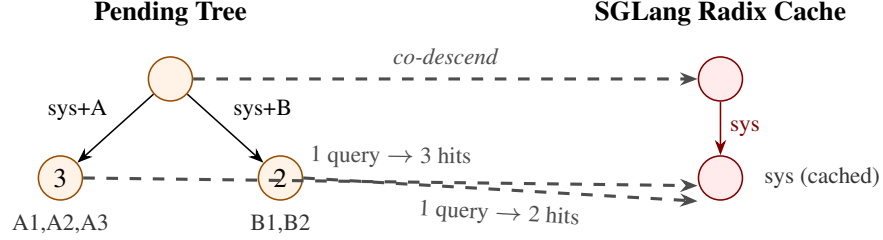
479 **Cost.** On SGLang, each pending edge incurs one cache co-traversal step; total is $O(C \times D)$ across
 480 C pending tree edges of average depth D , vs. $O(N \times D)$ for stock LPM—an amortization factor of
 481 N/C (large when sharing is high). On vLLM, the LPM lookup itself is not amortized: $O(N \times b)$
 482 for N `find_longest_cache_hit` calls (b = block-hash chain length), matching stock APC’s per-
 483 request cost; the additional $O(N)$ pending-tree walk for cluster signals is dwarfed by the LPM probes.
 484 Algorithm 1 (§3.1) details the SGLang procedure.

485 A.1.5 cLPM Scheduling Algorithm

486 Algorithm 2 details the per-cycle scheduling routine described in §3.2: dual-walk (or per-request LPM
 487 on vLLM) for $h(r)$, pending-tree lookups for cluster signals, lexicographic sort of two independent
 488 lane keys (cache-locality lane keyed by Eq. 1; fairness lane keyed by arrival), and stride-interleave
 489 between them.

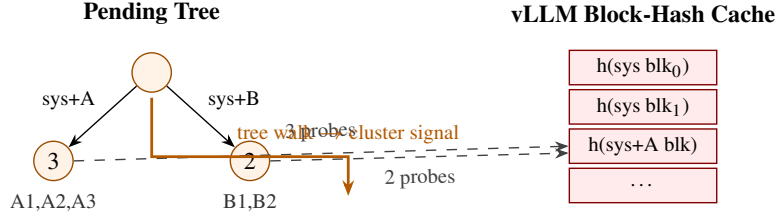
490 **Cost.** Dual-walk: $O(C \times D)$ on SGLang. On vLLM, the cluster signal is read directly from the
 491 pending tree via `all_cluster_info` ($O(N)$) and per-request LPM is satisfied by one $O(b)$ probe
 492 each into the engine’s block-hash index (b = block-hash chain length), totaling $O(N \times b)$ for the LPM
 493 lookup; the structural pending-tree walk used by SGLang is unnecessary because vLLM exposes a
 494 direct flat-cache lookup. Per-request `score/cluster` reads are $O(1)$ (computed once via batched
 495 Rust calls). Sort: $O(N \log N)$. Stride interleave: $O(N)$. Total per cycle: $O(C \times D + N \log N)$ on
 496 SGLang; $O(N \times b + N \log N)$ on vLLM.

497 **Guard.** The pending tree exposes `has_sharing()` (an $O(\# \text{root-children})$ check); when integration
 498 is configured to skip cLPM on no-sharing queues, Lines 1–16 are bypassed and the engine’s stock
 499 order is returned. Empirically PEEK matches FCFS within $\pm 1.5\%$ on no-sharing workloads (§3.3,
 500 §4.4), driven by both this guard and SGLang’s stock LPM-fallback at queue depth > 128 .



Stock LPM: 5 queries ($O(N \times D)$). PEEK: 2 queries ($O(C \times D)$).

(a) Dual-walk on SGLang: tree-vs-tree co-descent amortizes lookups across cluster siblings.



Stock vLLM: 5 hash probes, no cluster signal. PEEK: 5 probes + tree walk surfaces clusters for cLPM.

(b) Dual-walk on vLLM: per-request hash probes are unavoidable, but the pending-tree walk surfaces the cluster signal that vLLM's flat hash cache cannot.

Figure 6: Dual-walk on (a) SGLang and (b) vLLM. SGLang's radix cache permits tree-vs-tree co-descent, collapsing N per-request LPM lookups into C per-cluster lookups. vLLM's flat block-hash cache forces one probe per request, but the pending-tree walk still extracts cluster information for the scheduler downstream.

501 A.1.6 Eviction Motivation Example

502 Even with good scheduling, uninformed LRU eviction can undo prefix sharing. Consider three
 503 prefix groups A, B, C with a cache holding two prefixes. Wave 1 processes A then B, leaving
 504 both cached (B most recently used). Wave 2 needs A and C. When C arrives and the cache is full,
 505 LRU evicts A—the least recently accessed—but A is exactly what wave 2 needs next, forcing full
 506 recomputation. A queue-aware policy inspects pending requests, sees A is needed and B is not, and
 507 evicts B instead—preserving A for an immediate hit.



(a) LRU evicts A (least recent), but A is needed next. (b) Queue-aware: evicts B (not needed), preserves A.

Figure 7: LRU vs. queue-aware eviction. After wave 1 caches A and B, wave 2 needs A and C. LRU (a) evicts A—the least recently used—forcing recomputation. Queue-aware eviction (b) inspects pending requests, preserves A, and evicts the unreferenced B.

Algorithm 2 cLPM Per-Cycle Scheduling

Require: Waiting queue $\mathcal{R}$, pending tree T , engine cache $\mathcal{C}$, cache-lane share α , warm threshold τ , deprio threshold θ

Ensure: Admission order over $\mathcal{R}$

```
1:  $H \leftarrow \text{dual\_walk}(T, \mathcal{C})$   $\{H[r] = h(r) \text{ for each } r \text{ (Alg. 1)}\}$ 
2:  $S \leftarrow \text{compute\_req\_scores}(T)$ ;  $K \leftarrow \text{all\_cluster\_info}(T)$  {batched Rust calls}
3:  $\text{seen} \leftarrow \emptyset$ 
4: for each  $r \in \mathcal{R}$  in arrival order do
5:   if  $H[r] > \tau$  then
6:      $\text{section}(r) \leftarrow 0$  {warm}
7:   else
8:      $p \leftarrow \text{prompt}(r)[\theta]$ 
9:     if  $p \in \text{seen}$  then
10:       $\text{section}(r) \leftarrow 2$  {cold sibling (deprio tail)}
11:    else
12:       $\text{section}(r) \leftarrow 1$ ;  $\text{seen} \leftarrow \text{seen} \cup \{p\}$  {cold pioneer}
13:    end if
14:  end if
15:   $\text{score}(r) \leftarrow S[r]$ ;  $|\text{cluster}(r)| \leftarrow K[r].\text{size}$ 
16: end for
17:  $\mathcal{R}_A \leftarrow \text{sort}(\mathcal{R}, \text{key} = \text{Eq. 1})$  {cache-locality lane}
18:  $\mathcal{R}_B \leftarrow \text{sort}(\mathcal{R}, \text{key} = (\text{section}, \text{arrival}, -H))$  {fairness lane}
19: if  $\text{dynamic\_lane}$  then
20:    $\alpha \leftarrow 1 - \text{EMA}_{\beta=0.3}(\text{clamp}(0.15 + 0.5 \cdot \text{singleton\_frac} + 0.3 \cdot \text{age\_pressure}, 0.1, 0.6))$ 
21: end if
22: return  $\text{stride\_interleave}(\mathcal{R}_A, \mathcal{R}_B, \alpha)$  {cache-locality lane gets share  $\alpha$  (default 0.7)}
```

Table 4: Hardware, model, and dataset assignment per workload.

W	GPUs	Model (bf16)	Dataset
W1	1×NVIDIA H100 80 GB	Qwen2.5-32B-Instruct	LooGLE long-document corpus: 100 group prompts (~1024 tokens each), Zipf- $\alpha=1.0$ sampling
W2	1×NVIDIA H100 80 GB	Qwen2.5-32B-Instruct	LooGLE long-document corpus: 40 group prompts (~8192 tokens each), Zipf- $\alpha=1.0$ sampling
W3	2–4×NVIDIA H100 80 GB (TP=2; DP=1 / DP=2)	Llama-3.1-70B-Instruct	Multi-GPU scaling of W1 and W2: cell B mirrors W2 (long-prefix RAG), cell C mirrors W1 (shared-prompt chat)
W4	1×NVIDIA H100 80 GB	Mistral-Small-24B-Instruct-2501 (matches Mooncake trace)	Mooncake <i>conversation_trace</i> (FAST’25) with within-session prefix accumulation; <i>agentic_only</i> and <i>agentic_shared</i> scenarios
W5	1×NVIDIA H100 80 GB	Gemma-2-27B-it	LMSYS arena chat (heterogeneous singletons): <i>cell_C_long</i> (512–4096 tokens) and <i>cell_C_short</i> (32–1024 tokens)

A.2 PEEK Experiments

A.2.1 Experimental Setup Tables

A.2.2 Workload-to-Production-Scenario Mapping

A.2.3 W1 Full-Lattice Heavy-Load Tables

The three tables below report W1’s full 7-policy lattice on heavy load: throughput, cache hit, mean TTFT, and mean E2E on SGLang (Table 10) and vLLM (Table 11); decode-side mean TPOT and ITL on both engines (Table 12). The body’s W1 narrative (§4.1) summarizes these via the figures and ablation paragraphs.

Table 5: Shared experiment setup across W1–W5.

Component	Configuration
Engines	SGLang 0.5.9, vLLM 0.19.1
Framework	PyTorch 2.9.1, Python 3.12.13, CUDA graphs <i>on</i>
Memory budget	SGLang <code>mem_fraction_static=0.88</code> ; vLLM <code>gpu_memory_utilization=0.9</code>
Seeds	42, 142, 242 (3 replicates per cell; reported numbers are means across seeds; $\pm$ values are seed standard deviation)
Warmup	First ~ 10 –20% of requests per cell (50–400 requests depending on cell size) are excluded from metric aggregation
Load levels	Two operating points per cell, calibrated by sustained queue depth: <i>moderate</i> maintains a queue length of ~ 60 –100 requests; <i>heavy</i> sustains ~ 150 –200
Policy lattice (SGLang)	B0 (FCFS+LRU), B1 (LPM+LRU); P_c1pm (cLPM only), P_full (cLPM + group-major admission + queue-aware eviction)
Policy lattice (vLLM)	B2 (FCFS+APC+LRU, strongest stock baseline); P2 (eviction only), P3 (cLPM only), P4 (cLPM + group-major), P5 (P4 + dynamic-lane fairness), P6 (P4 + queue-aware eviction, primary co-design), P7 (P5 + queue-aware eviction)

Table 6: Experiments and per-workload configurations. “Probe axis” names the design facet each workload stresses relative to the W1 driving scenario.

W	Probe axis	Cells	Configuration
W1	Oversubscription (driving)	A: 2 $\times$, B: 4 $\times$, C: 8 $\times$ (primary), D: 16 $\times$ KV pressure; moderate / heavy	Zipf- $\alpha=1.0$ prefix groups, Poisson arrivals, fixed decode=128. Full ablation lattice (B2, P2–P7).
W2	Long, decode (decode-dominant)	cell B (4 $\times$), moderate / heavy	40 documents $\times$ 8192-token prefixes; decode mix 10:128, 25:512, 30:1024, 25:2048, 10:4096 (mean ≈ 1460); KV footprint $\approx 7\times$ arena, eviction always firing.
W3	Multi-GPU scaling (Llama-3.1-70B, TP=2)	cell B (mirrors W2, RAG-like) and cell C (mirrors W1, chat-like) at DP=1 (2 $\times$ H100) and DP=2 (4 $\times$ H100); heavy	Same dataset patterns as W1 (cell C) and W2 (cell B), executed at 70B scale across two data-parallel topologies.
W4	Within-session prefix accumulation (Mooncake)	<code>agentic_only</code> , <code>agentic_shared</code> ; moderate / heavy	Mooncake <code>conversation_trace</code> (FAST’25), 4 rounds, 50 ms inter-turn. <code>agentic_shared</code> adds a 1402-token shared system prompt (Cursor / Copilot / Claude Code pattern).
W5	Singleton-dominated chat (no-regress)	<code>cell_C_long</code> (512–4096 tokens, weak residual overlap), <code>cell_C_short</code> (32–1024 tokens, no structural sharing); heavy	LMSYS arena chat singletons; lognormal length distribution; decode 64–256; Poisson arrivals; $N=1500$.

516 A.2.4 W1 Throughput and E2E Figures

517 These two figures complement the TTFT figure in the body (Figure 3) and the full-lattice tables in
518 Appendix A.2.3.

Table 7: W1 full configuration. Top: shared across all cells. Bottom: per-cell parameters and per-engine calibrated arrival rates. KV footprint is the peak working-set size; oversubscription is footprint relative to the $\sim 50\text{K}$ -token arena. Rates target moderate = queue p50 in [60,127], heavy ≥ 128 (the SGLang LPM-fallback boundary).

Shared across all W1 cells										
Hardware	1 $\times$ NVIDIA H100 80 GB (bf16)									
Model	Qwen2.5-32B-Instruct									
Engines	SGLang 0.5.9 (B1=LPM+LRU baseline); vLLM 0.19.1 (B2=FCFS+APC+LRU baseline)									
Dataset	LooGLE long-document corpus, prefix tokens taken from documents									
Group sampling	Zipf- $\alpha=1.0$ over G shared-prompt groups									
Arrival process	Poisson (inter-arrival $\sim \text{Exp}(\lambda)$)									
Decode	Fixed 128 tokens per request									
Concurrency cap	vLLM: moderate=100, heavy=210; SGLang: 256									
Seeds	42, 142, 242 (3 replicates per cell; reported numbers are means across seeds)									
Warmup	First $N \cdot 0.2$ requests excluded from metrics									
Policies	B1 (SGLang) / B2 (vLLM); P2 (eviction-only); P3 (cLPM); P4 (cLPM+group-major); P5 (P4+dyn-lane); P6 (P4+queue-aware eviction, primary co-design); P7 (P5+queue-aware eviction)									

Cell	G	prefix		warmup		KV (tok)	over- sub.	SGLang (req/s)		vLLM (req/s)	
		(tok)	N	(req)				mod.	heavy	mod.	heavy
A	100	1024	1000	200		102 K	$2\times$	40	60	20	30
B	200	1024	2000	400		205 K	$4\times$	35	80	20	30
C	100	4096	1000	200		410 K	$8\times$ (primary)	4	8	5	10
D	200	4096	2000	400		820 K	$16\times$	12	24	8	15

Table 8: W1 cells mapped to production deployment classes. Cell C is designated primary as the most directly representative of PEEK’s target deployment.

Cell	Production analog	Role / question this cell answers
A (2 $\times$)	SMB chatbot, single-tenant SaaS (e.g. Intercom Fin AI) where the KV cache mostly fits	“Does PEEK matter at all when KV mostly fits?” — near-baseline reference
B (4 $\times$)	Multi-tenant SaaS / mid-size prompt library (Jasper-style content templates; multi-vertical chat platforms aggregating ~ 200 small tenants)	“Does PEEK help the typical SaaS deployment in steady state?” — common-case production test
C (8 $\times$, primary)	Complex prompts at scale: ChatGPT custom GPTs, Claude projects with tool-using assistants, Cursor / Copilot workspace prompts with file/repo context, enterprise platforms with ~ 100 tool-using personas	“Does PEEK help ChatGPT-class deployments under realistic peak pressure?” — the workload the design directly targets
D (16 $\times$)	Large multi-tenant agent platform under flash / extreme load: launch-day API peak, viral-spike chat platform, multi-vertical agent platform during launch	“Does PEEK hold up under flash / overload, or collapse along with everything else?” — stress test

Table 9: W1–W5 mapping to production deployment classes and serving hardware.

W	Class	Production scenarios	GPUs
W1	Shared-prompt / templated apps	SMB chatbots (Intercom Fin), multi-tenant SaaS prompt libraries (Jasper-style), custom-GPT / Claude-projects at scale, enterprise multi-tenant agent platforms under flash load	1×H100
W2	Long-document RAG (decode-dominant)	Enterprise RAG over document corpora (legal, medical, finance); long-form chat-over-doc; reasoning-adjacent synthesis	1×H100
W3	Multi-GPU 70B serving for shared-prompt + RAG	Production deployments of W1- and W2-class workloads on Llama-3.1-70B-class models scaled across DP×TP topologies (single-replica TP=2 and dual-replica DP=2×TP=2)	2–4×H100 (TP=2; DP=1 / DP=2)
W4	Agentic bursts / programmatic serving	Tool-calling assistants (Cursor, Copilot, Claude Code), RAG pipelines with cite-check, reasoning chains (ToT / SoT), computer-use agents, back-end LLM enrichment pipelines	1×H100
W5	Heterogeneous chat (no-regress)	Default ChatGPT / Claude / Gemini chat, open-source hosted chat (HuggingChat, LMSYS Arena), general-purpose inference APIs without enforced templating	1×H100

Table 10: W1 heavy load on **SGLang**: throughput, cache hit rate, mean TTFT, and mean E2E. Means across 3 seeds. PE = PEEK’s queue-aware eviction (demand_cluster); GM = group-major admission; DL = dynamic-lane fairness. cLPM alone is the cluster-LPM scheduler with stock LRU eviction (no group-major, no dynamic-lane).

Cell	Metric	LPM+LRU	LPM+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	tput (req/s)	7.43	7.36	14.46	14.80	14.98	14.89	15.03
	cache hit (%)	65.8	65.4	93.7	93.6	93.5	93.5	93.6
	TTFT (s)	42.6	43.1	11.0	10.8	10.4	10.5	10.3
	E2E (s)	47.8	48.3	16.9	16.4	16.0	16.2	15.9
B (4×)	tput (req/s)	6.73	6.70	13.74	14.34	14.49	14.66	14.55
	cache hit (%)	59.2	58.8	91.9	93.3	93.3	93.3	93.2
	TTFT (s)	55.0	55.3	16.5	15.4	14.9	14.9	15.0
	E2E (s)	60.3	60.8	22.5	21.1	20.8	20.8	20.8
C (8×)	tput (req/s)	1.42	1.41	3.72	3.75	3.71	3.75	3.73
	cache hit (%)	38.3	38.3	88.3	89.4	89.2	89.2	88.8
	TTFT (s)	233.2	234.0	36.9	32.7	31.5	32.6	29.7
	E2E (s)	238.2	239.0	42.6	38.3	37.2	38.3	35.3
D (16×)	tput (req/s)	1.22	1.20	4.28	4.42	4.36	4.44	4.41
	cache hit (%)	30.8	30.9	91.0	92.2	92.2	92.4	92.1
	TTFT (s)	335.5	340.2	62.5	59.6	60.3	59.9	59.7
	E2E (s)	340.6	345.4	69.1	66.1	66.7	66.3	66.2

Table 11: W1 heavy load on **vLLM**: throughput, cache hit rate, mean TTFT, and mean E2E. All vLLM cells run with Automatic Prefix Caching (APC) enabled. Means across 3 seeds. Column abbreviations: PE = PEEK’s queue-aware eviction; GM = group-major admission; DL = dynamic-lane fairness.

Cell	Metric	FCFS(APC)+LRU	FCFS(APC)+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	tput (req/s)	8.50	8.34	11.58	13.14	11.41	12.96	13.04
	cache hit (%)	66.3	66.1	87.5	86.4	86.7	86.8	86.7
	TTFT (s)	14.0	14.4	6.0	4.2	6.1	4.3	4.2
	E2E (s)	19.3	19.9	11.1	10.4	11.2	10.7	10.4
B (4×)	tput (req/s)	7.52	7.31	10.83	11.76	10.65	11.56	11.57
	cache hit (%)	58.1	57.9	83.7	81.1	83.5	81.7	81.7
	TTFT (s)	17.1	17.7	8.0	6.5	7.8	6.5	6.4
	E2E (s)	22.4	23.1	13.2	12.6	13.1	12.8	12.5
C (8×)	tput (req/s)	1.68	1.82	4.90	4.36	4.80	4.24	4.25
	cache hit (%)	37.7	38.0	86.5	86.2	86.2	85.8	86.3
	TTFT (s)	97.8	86.6	16.9	21.8	18.5	22.6	22.4
	E2E (s)	102.6	91.5	22.4	27.2	23.8	28.7	28.5
D (16×)	tput (req/s)	1.54	1.46	3.04	3.60	2.96	3.57	3.52
	cache hit (%)	32.8	31.1	81.8	78.9	80.9	79.2	79.0
	TTFT (s)	106.3	111.1	39.4	35.0	40.5	34.4	33.2
	E2E (s)	111.3	116.3	44.4	40.4	45.6	40.4	39.0

519 **A.2.5 W2 Heavy and Moderate Load Tables**520 **A.2.6 W2 Throughput and E2E Figures**521 **A.2.7 W3 Heavy-Load Tables (DP=1 and DP=2)**522 **A.2.8 W3 Throughput and E2E Figures (DP=1 and DP=2)**

18

523 **A.2.9 W4 Heavy-Load Tables**524 **A.2.10 W4 Figures**

Table 12: W1 heavy load decode-side metrics: mean TPOT and mean ITL on both engines. Means across 3 seeds. TPOT and ITL agree to within 0.1 ms in every cell. All vLLM cells run with APC enabled. Column abbreviations: PE = PEEK’s queue-aware eviction; GM = group-major admission; DL = dynamic-lane fairness.

SGLang								
Cell	Metric	LPM+LRU	LPM+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	TPOT (ms)	45.9	46.7	52.0	49.4	49.6	50.9	50.5
	ITL (ms)	45.8	46.5	51.9	49.3	49.6	50.9	50.4
B (4×)	TPOT (ms)	46.9	47.6	52.4	50.2	51.4	52.1	50.5
	ITL (ms)	46.9	47.7	52.3	50.2	51.3	52.0	50.5
C (8×)	TPOT (ms)	44.4	44.4	49.9	49.4	49.9	50.2	49.1
	ITL (ms)	44.5	44.5	50.0	49.4	49.9	50.2	49.1
D (16×)	TPOT (ms)	44.1	45.0	56.0	55.5	55.4	54.9	55.6
	ITL (ms)	44.2	45.0	56.0	55.5	55.3	55.0	55.7

vLLM								
Cell	Metric	FCFS(APC)+LRU	FCFS(APC)+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	TPOT (ms)	43.5	44.3	44.3	50.0	44.8	51.7	50.3
	ITL (ms)	43.5	44.3	44.2	50.0	44.6	51.4	50.2
B (4×)	TPOT (ms)	42.2	43.2	44.8	49.4	45.1	51.1	49.8
	ITL (ms)	42.2	43.2	44.5	49.4	45.0	51.0	49.8
C (8×)	TPOT (ms)	39.5	39.0	42.7	43.1	42.4	49.3	49.3
	ITL (ms)	39.4	39.0	43.0	43.1	42.4	49.3	49.3
D (16×)	TPOT (ms)	40.5	41.8	42.1	43.7	42.5	48.4	46.2
	ITL (ms)	40.5	41.8	42.1	43.7	42.5	48.3	46.7

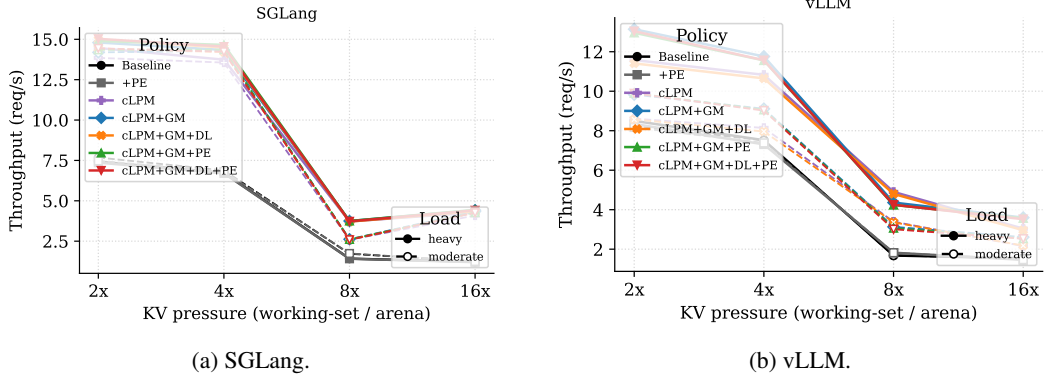


Figure 8: W1 throughput vs. KV pressure. Each panel overlays both load levels: solid lines with filled markers = heavy, dashed lines with hollow markers = moderate.

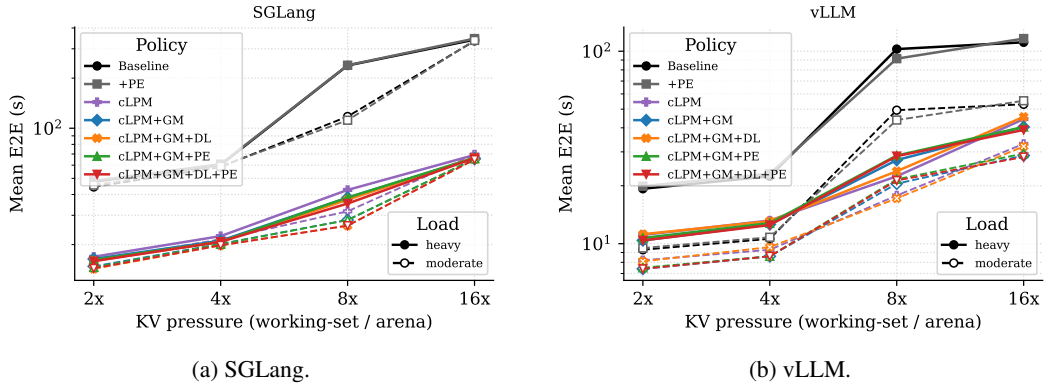


Figure 9: W1 mean E2E latency vs. KV pressure (log y-axis). Conventions as in Figure 3. Same pattern: baselines climb, PEEK stays bounded.

531 this by replacing the per-cycle pending tree of PEEK-online with a one-shot, full-batch construction,
532 executed entirely as client-side preprocessing through the engine’s public API.

Table 13: W2 on **SGLang** (cell B, $7 \times$ KV pressure): throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. Means across 3 seeds. Both rates run slightly above r_{sat} (errored counts in the single digits), so throughput represents a saturated regime; cache-hit and TTFT deltas remain interpretable.

Load	Metric	LPM+LRU	cLPM+GM+DL+PE
moderate	tput (req/s)	0.289	0.330
	cache hit (%)	62.4	79.4
	TTFT (s)	391.1	178.8
	TPOT (ms)	32.6	31.8
	E2E (s)	419.2	206.8
heavy	tput (req/s)	0.283	0.357
	cache hit (%)	60.0	83.2
	TTFT (s)	543.4	212.2
	TPOT (ms)	31.9	33.4
	E2E (s)	571.5	241.6

Table 14: W2 on **vLLM** (cell B, $7 \times$ KV pressure): throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. All cells run with APC enabled. Means across 3 seeds. errored=0 across all cells.

Load	Metric	FCFS(APC)+LRU	cLPM+GM+DL+PE
moderate	tput (req/s)	0.114	0.114
	cache hit (%)	29.4	33.0
	TTFT (s)	28.4	8.3
	TPOT (ms)	27.4	27.8
	E2E (s)	51.4	31.6
heavy	tput (req/s)	0.143	0.151
	cache hit (%)	29.5	40.1
	TTFT (s)	90.7	24.7
	TPOT (ms)	28.8	31.2
	E2E (s)	114.6	50.5

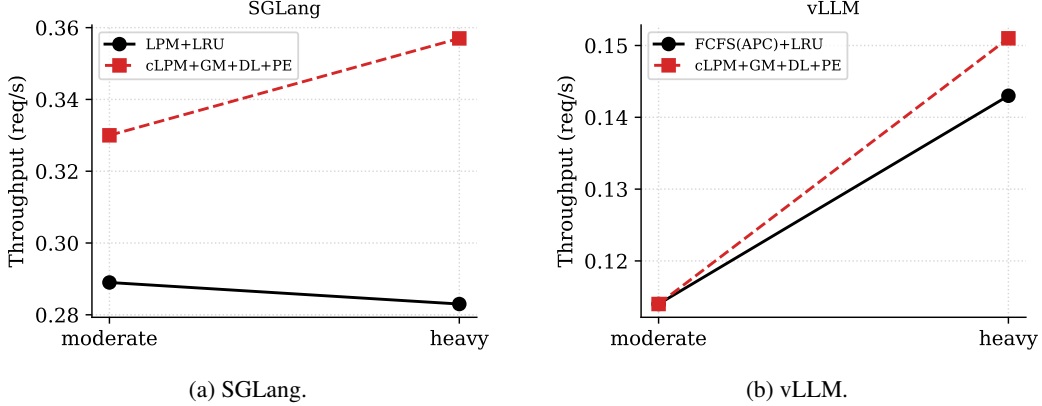


Figure 10: W2 throughput on cell B, both engines and both load levels.

533 **Phase 1: Prefix trie DFS reordering.** PEEK-offline builds a token-level prefix trie over all N
534 submitted requests by inserting the first D tokens of each prompt (default $D=128$); each node
535 records the requests terminating there and the subtree request count. A depth-first traversal that visits
536 children in descending subtree-count order emits a permutation π in which prefix-sharing requests
537 are adjacent, even when the engine cache is cold. Construction is $O(N \cdot D)$ and traversal is $O(|T|)$
538 with $|T| \ll N \cdot D$ under sharing; at $N=5,000$ the entire reorder completes in <50 ms on a single
539 CPU core.

540 **Phase 2: Shadow LRU cache and wave-aware refinement.** The Phase 1 ordering is parti-
541 tioned into *waves* of size W approximating the engine’s running batch. A client-side shadow LRU
542 cache, hashed identically to vLLM/SGLang ($k_i = \text{hash}(k_{i-1}, \text{tuple}(s[iB:(i+1)B]))$, $B=16$), sim-
543 ulates per-block residency in $O(1)$ amortized time and is reconciled against the engine’s reported
544 `cached_tokens` after every completion. A second cache-aware DFS within each wave scores

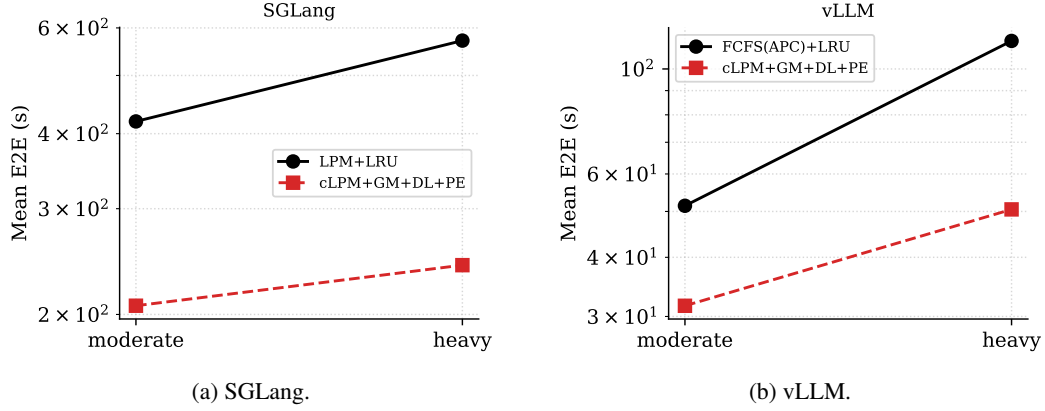


Figure 11: W2 mean E2E latency on cell B (log y-axis), both engines and both load levels.

Table 15: W3 heavy load on **SGLang** (Llama-3.1-70B, TP=2): throughput, cache hit, mean TTFT, mean TPOT, mean E2E at DP=1 (2×H100) and DP=2 (4×H100). Means across 3 seeds.

Cell	Metric	DP=1 (2×H100)		DP=2 (4×H100)	
		LPM+LRU	cLPM+GM+DL+PE	LPM+LRU	cLPM+GM+DL+PE
B (RAG-like)	tput (req/s)	0.464	1.237	1.043	1.689
	cache hit (%)	55.1	98.2	87.8	96.9
	TTFT (s)	324.2	85.3	180.7	91.4
	TPOT (ms)	32.4	32.8	31.9	32.8
	E2E (s)	330.1	91.3	192.6	103.6
C (chat-like)	tput (req/s)	2.896	6.022	7.757	13.634
	cache hit (%)	44.6	84.2	72.9	91.5
	TTFT (s)	46.6	15.7	28.7	10.2
	TPOT (ms)	47.6	44.3	37.0	36.7
	E2E (s)	50.0	18.9	33.4	14.9

Table 16: W3 heavy load on **vLLM** (Llama-3.1-70B, TP=2; APC enabled): throughput, cache hit, mean TTFT, mean TPOT, mean E2E at DP=1 and DP=2. Means across 3 seeds.

Cell	Metric	DP=1 (2×H100)		DP=2 (4×H100)	
		FCFS(APC)+LRU	cLPM+GM+DL+PE	FCFS(APC)+LRU	cLPM+GM+DL+PE
B (RAG-like)	tput (req/s)	0.241	1.072	0.478	2.152
	cache hit (%)	37.7	97.1	55.4	97.3
	TTFT (s)	559.6	83.2	384.4	54.3
	TPOT (ms)	36.0	65.9	42.1	93.9
	E2E (s)	572.7	103.4	399.6	83.8
C (chat-like)	tput (req/s)	1.798	3.728	4.040	11.440
	cache hit (%)	37.2	76.5	51.4	92.7
	TTFT (s)	75.8	24.9	60.5	12.3
	TPOT (ms)	32.1	48.1	32.8	43.7
	E2E (s)	79.9	31.0	64.7	17.9

children by

$$\text{key}(v) = \text{cache_frac}(v) \cdot 10^4 - \text{future_refs}(v) + \text{count}(v) \cdot 10^{-3},$$

strictly preferring already-cached groups (exploit), deferring groups demanded by later waves (preserve), and using subtree count only as a tiebreaker. A queue-aware eviction hook scans the remaining batch to derive per-block reference counts, evicting unreferenced blocks first and protecting referenced blocks via log-dampened depth-weighted priority to avoid ossification.

Guard conditions. Three cheap pre-checks skip all PEEK work when reordering cannot pay for itself: *duplication* (>50% exact-duplicate sequences), *coverage* (<10% of requests share ≥ 32 tokens), and *depth* (average sharing depth <64 tokens). When any guard fires, PEEK-offline reduces to FCFS at $O(1)$ overhead.

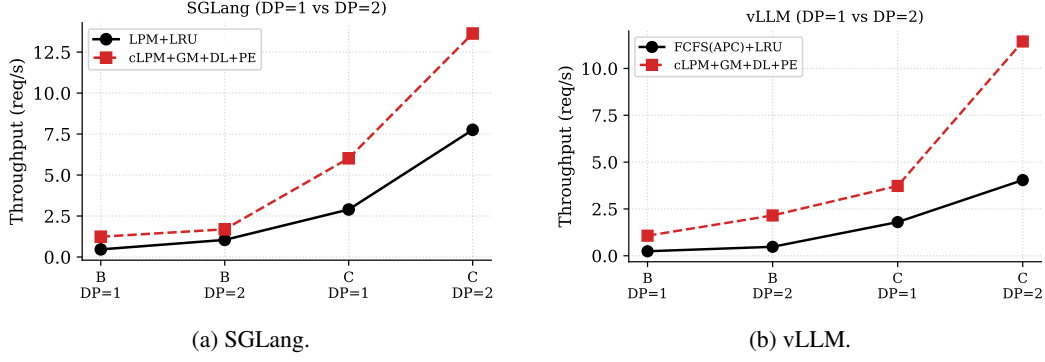


Figure 12: W3 throughput, DP=1 vs DP=2, heavy load.

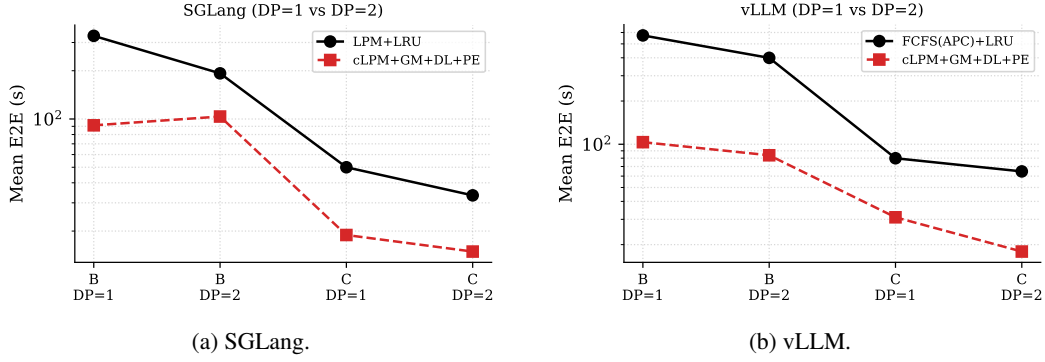


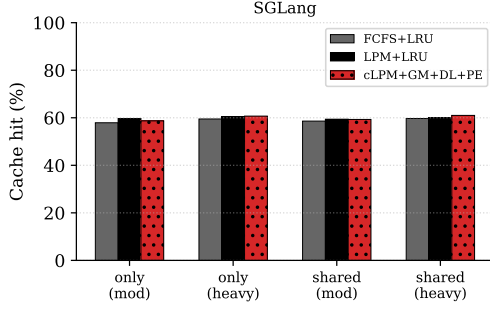
Figure 13: W3 mean E2E latency (log y-axis), DP=1 vs DP=2, heavy load.

Table 17: W4 heavy load on **SGLang**: throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. Means across 3 seeds.

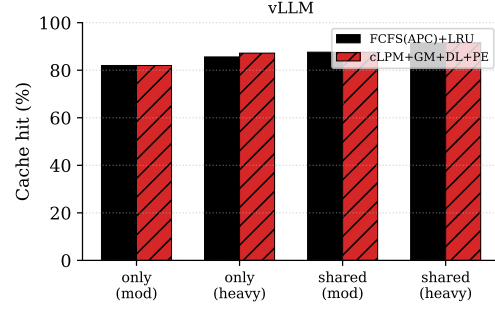
Scenario	Metric	FCFS+LRU	LPM+LRU	cLPM+GM+DL+PE
agentic_only	tput (req/s)	3.58	3.58	3.62
	cache hit (%)	59.5	60.4	60.7
	TTFT (ms)	1175	1129	1185
	TPOT (ms)	65.8	63.9	63.7
	E2E (s)	22.4	22.0	21.8
agentic_shared	tput (req/s)	3.59	3.58	3.62
	cache hit (%)	59.7	60.0	61.0
	TTFT (ms)	1181	1142	1157
	TPOT (ms)	66.3	65.1	62.8
	E2E (s)	22.4	22.2	21.6

Table 18: W4 heavy load on **vLLM**: throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. All cells run with APC enabled. Means across 3 seeds.

Scenario	Metric	FCFS(APC)+LRU	cLPM+GM+DL+PE
agentic_only	tput (req/s)	3.57	3.58
	cache hit (%)	85.6	87.2
	TTFT (ms)	178	167
	TPOT (ms)	22.8	22.5
	E2E (s)	5.40	5.31
agentic_shared	tput (req/s)	3.54	3.54
	cache hit (%)	91.5	91.5
	TTFT (ms)	171	173
	TPOT (ms)	22.8	22.9
	E2E (s)	4.78	4.79

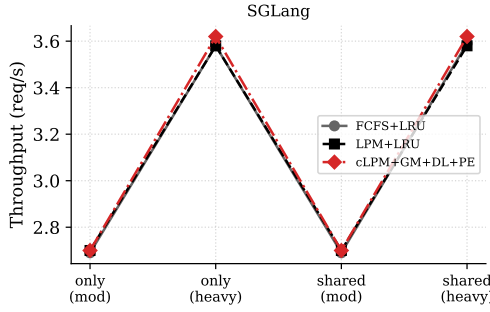


(a) SGLang.

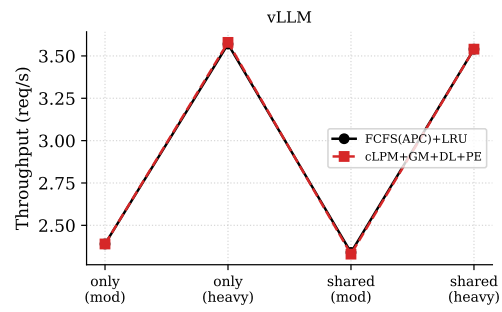


(b) vLLM.

Figure 14: W4 cache hit rate by scenario and load level, side-by-side per engine.

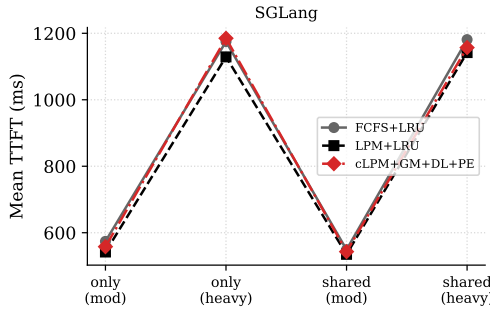


(a) SGLang.

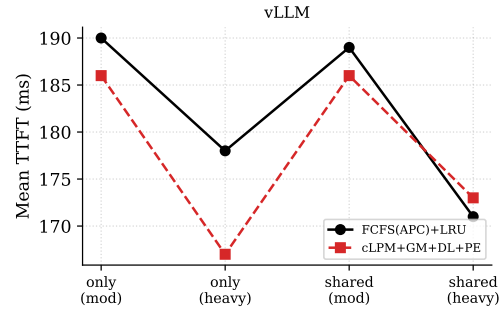


(b) vLLM.

Figure 15: W4 throughput by scenario and load level, side-by-side per engine.

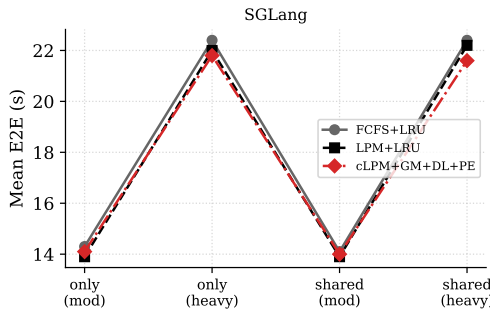


(a) SGLang.

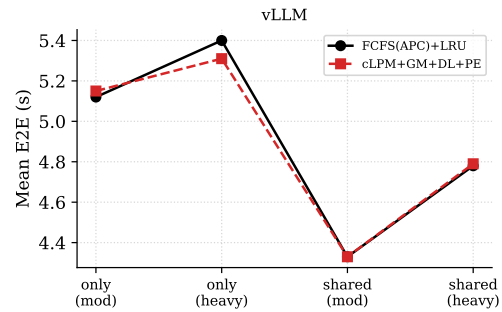


(b) vLLM.

Figure 16: W4 mean TTFT by scenario and load level, side-by-side per engine.



(a) SGLang.



(b) vLLM.

Figure 17: W4 mean E2E latency by scenario and load level, side-by-side per engine.

Table 19: W5 heavy load on **SGLang**: throughput, cache hit, mean TTFT, mean TPOT, mean E2E. Means across available seeds (3 seeds for cell_C_short; cell_C_long: seed 242 only). Cache hit computed from Prometheus snapshots (RadixAttention requires exact-prefix matches; with no shared system prompt, baseline cache hit is near zero).

Cell	Metric	LPM+LRU	cLPM+GM+DL+PE
C_long	tput (req/s)	7.83	8.20
	cache hit (%)	0.7	10.2
	TTFT (s)	21.7	19.4
	TPOT (ms)	52.2	52.7
	E2E (s)	28.3	26.0
C_short	tput (req/s)	21.89	21.68
	cache hit (%)	3.7	3.7
	TTFT (s)	3.6	3.6
	TPOT (ms)	54.6	54.9
	E2E (s)	10.05	10.14

Table 20: W5 on **vLLM**: throughput, cache hit, mean TTFT, mean TPOT, mean E2E. Means across 3 seeds. APC’s block-hash matching gives high reported cache hit on byte-recurring tokens across heterogeneous prompts (a property of the cache layer, not of structural sharing in the workload).

Cell	Metric	FCFS(APC)+LRU	cLPM+GM+DL+PE
C_long	tput (req/s)	18.33	22.27
	cache hit (%)	97.5	98.8
	TTFT (s)	4.25	1.61
	TPOT (ms)	73.6	74.3
	E2E (s)	12.45	9.98
C_short	tput (req/s)	39.63	39.70
	cache hit (%)	96.7	96.7
	TTFT (s)	0.39	0.35
	TPOT (ms)	43.9	44.0
	E2E (s)	5.69	5.66

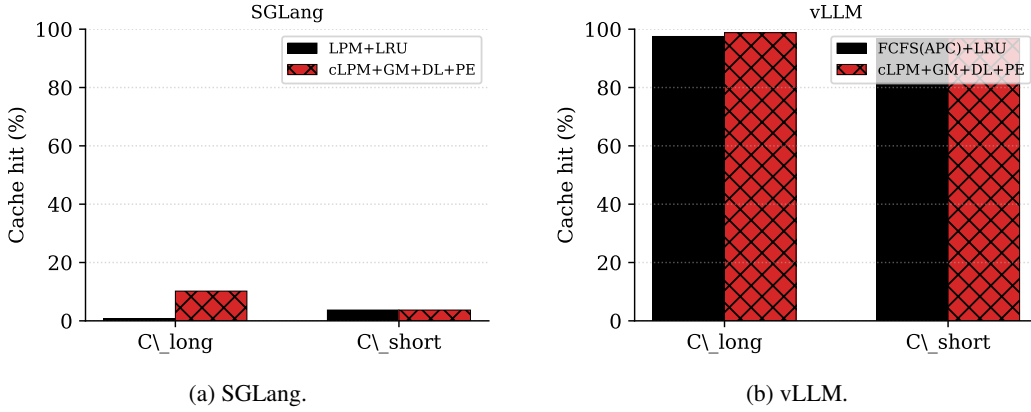


Figure 18: W5 cache hit rate by cell, both engines.

554 A.3.2 Experiment Results

555 PEEK-offline is evaluated on two configurations: *Single-GPU* (Qwen2.5-14B-Instruct, 1×H100
556 80 GB, FP16) and *Multi-GPU* (Qwen2.5-72B-Instruct, 2×H100, TP=2, FP8), both on SGLang v0.5.9
557 and vLLM v0.10.x at production-default memory (SGLang mem_fraction_static=0.80, vLLM
558 gpu_memory_utilization=0.90). Single-GPU uses six workloads (three high-sharing: shared
559 system prompts on LooGLE, mixed traffic, multi-turn ShareGPT chat; three low-sharing: few-shot
560 MMLU, code completion on HumanEval/MBPP, single-turn diverse). Multi-GPU uses six production
561 scenarios (Legal RAG, Long-Doc Summarization, Multi-Agent, Enterprise Search, Mixed Traffic,
562 plus a non-sharing control). Baselines are FCFS, LPM, and DFS-Weight with LRU/LFU on SGLang;
563 FCFS and APC with LRU on vLLM.

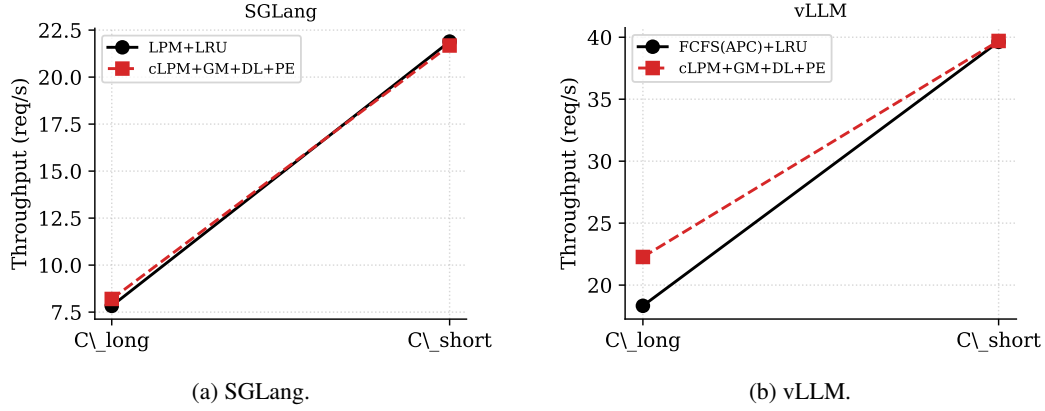


Figure 19: W5 throughput by cell, both engines.

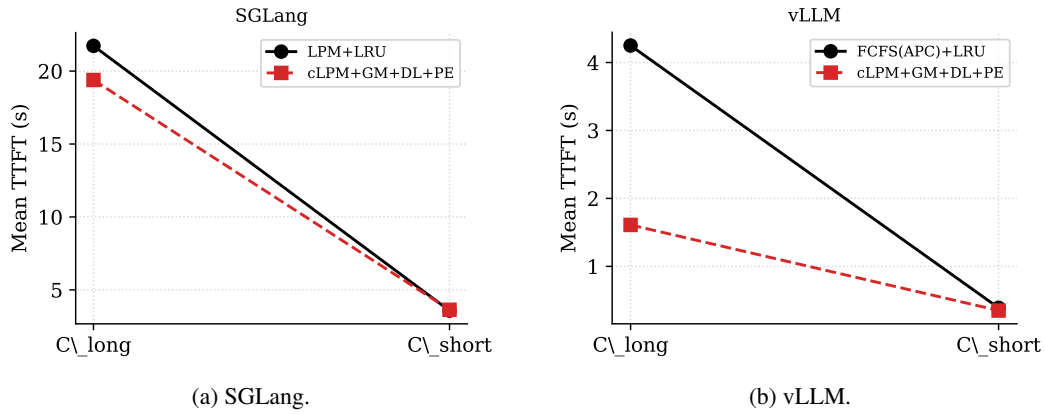


Figure 20: W5 mean TTFT by cell, both engines.

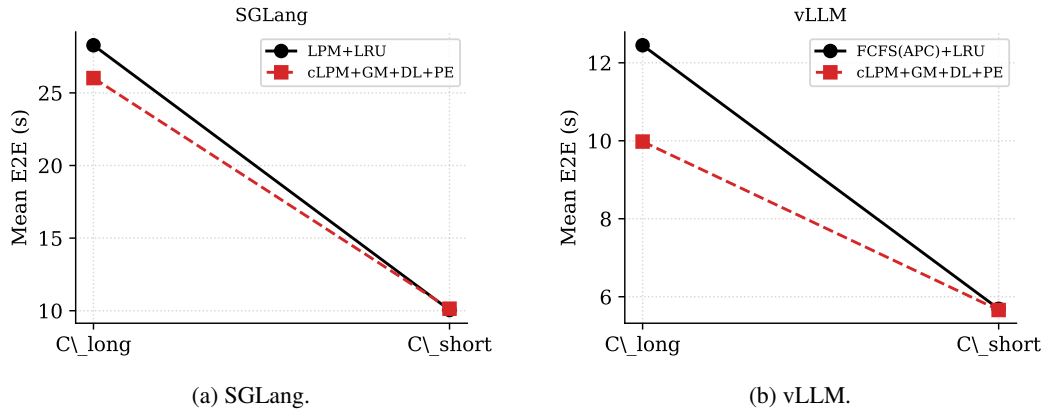


Figure 21: W5 mean E2E latency by cell, both engines.

564 **Single-GPU.** On the three high-sharing workloads, PEEK-offline beats the strongest baseline by
565 **1.1–1.6 $\times$** on SGLang (cache hits 72–84%) and **1.0–1.8 $\times$** on vLLM (71–85%). The largest gains
566 are on shared system prompts where 100 prefix groups arrive randomly interleaved (FCFS achieves
567 only 1% hits); the smallest are on multi-turn chat, where sequential conversation turns supply natural
568 adjacency (FCFS already at 50% hits). Prefill reduction accounts for 89–97% of total latency
569 improvement; the remainder is decode-side, an indirect benefit of freeing GPU compute sooner. On
570 the three low-sharing workloads, all policies are within 3% of each other—the guards correctly skip
571 reordering. P99/Mean latency ratios stay within 0.05 of the best baseline across all six workloads,
572 confirming no starvation.

573 **Multi-GPU.** Scaling to Qwen2.5-72B on $2\times$ H100, PEEK-offline beats the strongest baseline by
574 **1.6–2.4 $\times$** on SGLang (cache hits 90–96%) across five high-sharing scenarios and by **1.6–8.0 $\times$** on
575 vLLM (90–98%) across four scenarios. The largest gain is on Multi-Agent (16K shared prefixes), the
576 smallest on Mixed Traffic (moderate natural adjacency). Non-sharing controls remain within 5% of
577 baseline.

578 **Component ablation and scaling.** A reorder-only variant (DFS without shadow cache or eviction)
579 matches the full system on every workload, while eviction-only delivers no benefit in isolation—DFS
580 reordering supplies the temporal locality that queue-aware eviction then preserves. Sweeps confirm
581 gains scale monotonically with cache pressure: prefix length 512→4096 grows the gain from $1.0\times$
582 to **3.0–3.8 $\times$** ; group count 20→200 from $\sim 1\times$ to **2.2 $\times$** ; concurrency 16→256 stays in the **1.3–1.8 $\times$**
583 range. Under variable prefix lengths ($\sigma=0.0\text{--}0.6$) PEEK’s hit rate holds at 83–85% while baselines
584 degrade.

585 **Takeaway.** Across both single- and multi-GPU offline serving, queue-derived prefix structure is the
586 dominant lever for cache efficiency; the same insight that drives PEEK-online in this paper carries
587 over to the batch regime, where the per-cycle pending tree collapses to a one-shot batch trie but the
588 underlying co-design of prefix-grouped admission and queue-aware eviction is unchanged.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state PEEK’s three mechanisms (incremental pending radix tree, dual-walk LPM, cluster-aware admission/eviction with adaptive fairness lane) and the headline numbers; §4.1–§4.4 report the corresponding cache-hit, TTFT, E2E, and throughput results that match the abstract claims.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The conclusion notes that PEEK’s N/C amortization shrinks toward 1 in singleton-dominated traffic (with the `has_sharing` guard handling that regime; §3.3, §4.4); the cLPM sort still runs in Python; evaluation is on H100 only. Workloads cover five representative regimes; multi-modal and very-long-context settings are open.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This is a systems paper; it makes no formal theoretical claims requiring proof. Complexity statements (e.g., $O(C \times D)$ dual-walk, $O(D)$ insert/remove) are stated with the supporting algorithm in §3.1 and Appendix A.1.4.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: §4 and Appendix Tables 4, 5, 6 list engines (SGLang 0.5.9, vLLM 0.19.1), models, datasets, hardware, memory budget, seeds (42/142/242), warmup, and load levels. Algorithm 1 and Algorithm 2 fully specify PEEK’s procedures.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

695 (d) We recognize that reproducibility may be tricky in some cases, in which case
696 authors are welcome to describe the particular way they provide for reproducibility.
697 In the case of closed-source models, it may be that access to the model is limited in
698 some way (e.g., to registered users), but it should be possible for other researchers
699 to have some path to reproducing or verifying the results.

700 5. Open access to data and code

701 Question: Does the paper provide open access to the data and code, with sufficient instruc-
702 tions to faithfully reproduce the main experimental results, as described in supplemental
703 material?

704 Answer: [Yes]

705 Justification: PEEK’s source (Rust core + SGLang/vLLM Python integrations) will be
706 released; the abstract and conclusion both state “Code will be released.” All datasets used
707 (LooGLE, Mooncake conversation_trace, LMSYS arena chat) are public; models (Qwen2.5,
708 Llama-3.1, Mistral-Small, Gemma-2) are publicly available under their respective licenses.

709 Guidelines:

- 710 • The answer [N/A] means that paper does not include experiments requiring code.
- 711 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
712 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 713 • While we encourage the release of code and data, we understand that this might not
714 be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not
715 including code, unless this is central to the contribution (e.g., for a new open-source
716 benchmark).
- 717 • The instructions should contain the exact command and environment needed to run to
718 reproduce the results. See the NeurIPS code and data submission guidelines ([https:
719 //neurips.cc/public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 720 • The authors should provide instructions on data access and preparation, including how
721 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 722 • The authors should provide scripts to reproduce all experimental results for the new
723 proposed method and baselines. If only a subset of experiments are reproducible, they
724 should state which ones are omitted from the script and why.
- 725 • At submission time, to preserve anonymity, the authors should release anonymized
726 versions (if applicable).
- 727 • Providing as much information as possible in supplemental material (appended to the
728 paper) is recommended, but including URLs to data and code is permitted.

729 6. Experimental setting/details

730 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
731 rameters, how they were chosen, type of optimizer) necessary to understand the results?

732 Answer: [Yes]

733 Justification: §4 (Table 2) defines policy labels; Appendix Tables 4–6 give per-workload
734 model, hardware, dataset, cell parameters, and load levels; §3.2 specifies cLPM hyperparam-
735 eters ($\theta=32$, τ , default $\alpha=0.7$, EMA $\beta=0.3$, clamp $[0.10, 0.60]$).

736 Guidelines:

- 737 • The answer [N/A] means that the paper does not include experiments.
- 738 • The experimental setting should be presented in the core of the paper to a level of detail
739 that is necessary to appreciate the results and make sense of them.
- 740 • The full details can be provided either with the code, in appendix, or as supplemental
741 material.

742 7. Experiment statistical significance

743 Question: Does the paper report error bars suitably and correctly defined or other appropriate
744 information about the statistical significance of the experiments?

745 Answer: [Yes]

Justification: All reported numbers are means across 3 seeds (42, 142, 242) after warmup; Appendix Table 5 states “ $\pm$ values are seed standard deviation,” and per-workload tables in the appendix include seed-level dispersion where applicable.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix Table 4 lists per-workload hardware (1×H100 80 GB for W1/W2/W4/W5, 2–4×H100 80 GB with TP=2 and DP=1/2 for W3); Appendix Table 5 lists memory budgets (mem_fraction_static=0.88 SGLang, gpu_memory_utilization=0.9 vLLM) and Python/PyTorch/CUDA versions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn’t make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work is a serving-system optimization (scheduling and eviction policy); it involves no human subjects, no scraped or sensitive data, and no model training. All datasets and models used are public and properly licensed.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: PEEK improves KV-cache utilization in LLM serving, reducing GPU/energy cost per request—a positive efficiency impact. The same efficiency lowers the cost of operating large-scale LLM deployments, which could indirectly enable applications with negative externalities; however, PEEK is policy-level and applies orthogonally to any deployment, so the impact mirrors that of the underlying LLM rather than introducing new harms.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: PEEK is a serving-policy framework—no pre-trained models, generators, or scraped datasets are released. Released artifacts (Rust/Python serving-policy code) carry no foreseeable misuse risk.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All engines, models, and datasets are cited and used under their public licenses: SGLang and vLLM (Apache 2.0); Qwen2.5-32B/14B/72B and Mistral-Small (Apache 2.0); Llama-3.1-70B (Llama 3.1 Community License); Gemma-2-27B (Gemma terms of use); LooGLE (research-use); Mooncake conversation_trace (FAST'25 release); LMSYS arena chat (CC BY-NC).

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The PEEK code base (~1.6 kLOC Rust core + ~800 LOC per-engine Python integration; Appendix A.1.2) will be released with documentation describing the activation flags (PEEK_SCHEDULER, PEEK_EVICTION, etc.), engine-version compatibility (SGLang 0.5.9, vLLM 0.19.1), and per-workload reproduction scripts.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects, so no IRB review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs in this paper are the systems being served (Qwen2.5, Llama-3.1-70B, Mistral-Small, Gemma-2)—i.e., the workload, not a methodological component. PEEK's scheduling and eviction policies do not use LLMs; complexity, mechanism, and evaluation are LLM-agnostic.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.